```
// BootRunner.cs
using System.Collections;
using Sy.BloomHaven.Codex;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Guide;
using Sy.BloomHaven.Orders;
using Sy.BloomHaven.Save;
using Sy.BloomHaven.Share;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.BloomHaven.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S19_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            SaveHost.Ensure(); // T007 slice b: load + offline settle before any page reads state
            GardenView.InstallOnSceneLoad(); // T003 slice B: garden renders once S19_Main is live
            OrderBoardView.InstallOnSceneLoad(); // T004 slice B: order board page (S02) on S19_Main
            ShareView.InstallOnSceneLoad(); // T006: share page (S04) on S19_Main, panel starts hidden
            CodexView.InstallOnSceneLoad(); // T010-b: codex + boosts page (S03) on S19_Main, panel starts hidden
            NewbieGuide.InstallOnSceneLoad(); // T006: newbie guide, last so the garden system is live
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
```

```
            while (op != null &amp;amp;&amp;amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.BloomHaven.Boot
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
```

```
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.BloomHaven.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
```

```
    {
        if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
        ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
        Logger.Info("BOOT", "mock services registered (SY_MOCK)");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
```

```
            =&gt; new FontSet("Fonts/G19Main", "Fonts/G19Main", "Fonts/G19Main");
    }
    public sealed class MockAdsService : IAdsService
    {
        public event Action&lt;AdLifecycle&gt; OnAdEvent;
        public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
            =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
        public void ShowInterstitial(string pointId) { }
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// AudioHost.cs
using UnityEngine;
namespace Sy.BloomHaven.Audio
{
    /// &lt;summary&gt;
    /// Game-wide audio face (T008): one persistent host owns the SFX one-shot
    /// channel and the looping BGM channel. Hook entry points are static so
    /// views fire them without holding references; the clip resolve goes
    /// through G19AudioRuntime and a missing clip stays SILENT (no placeholder
    /// bleeps - assets arrive with the T001 TJ batch). Hook counters advance
    /// even when clips are missing so gates can assert wiring while the audio
    /// batch is still pending; the actual PlayOneShot/Play only runs in play
    /// mode.
    /// &lt;/summary&gt;
    public sealed class AudioHost : MonoBehaviour
    {
        public int BloomHookFired;
        public int PlantHookFired;
        public int OrderChimeFired;
        public int OrderRefreshFired; // T010-b: boost-refresh clip slot counter
        public int HarvestHookFired;  // X1079: petal-pluck harvest counter
        public int ShareHookFired;    // X1079: share-confirmation counter
        public int BgmStartCalls;
        public bool BgmClipPresent; // honest face: resolved at the last StartBgm call
        public static readonly string HostName = "G19Audio";
        static AudioHost _instance;
        AudioSource _sfx;
        AudioSource _bgm;
        public static AudioHost EnsureHost()
        {
            if (_instance != null) return _instance;
            var existing = (AudioHost)FindObjectOfType(typeof(AudioHost));
            if (existing != null)
            {
```

```
        _instance = existing;
        return _instance;
    }
    var go = new GameObject(HostName, typeof(AudioHost));
    _instance = go.GetComponent&lt;AudioHost&gt;();
    _instance._sfx = go.AddComponent&lt;AudioSource&gt;();
    _instance._sfx.playOnAwake = false;
    _instance._bgm = go.AddComponent&lt;AudioSource&gt;();
    _instance._bgm.playOnAwake = false;
    _instance._bgm.loop = true;
    if (Application.isPlaying) DontDestroyOnLoad(go); // survive S19 reloads
    return _instance;
}
// ----- hook trio (T001 spec) + BGM -----
public static void PlayBloomHook()
{
    var host = EnsureHost();
    host.BloomHookFired++;
    host.PlayOneShotSafe(AudioStems.SfxBloom, 0.9f);
}
public static void PlayPlantThud()
{
    var host = EnsureHost();
    host.PlantHookFired++;
    host.PlayOneShotSafe(AudioStems.SfxPlant, 0.8f);
}
public static void PlayOrderChime()
{
    var host = EnsureHost();
    host.OrderChimeFired++;
    host.PlayOneShotSafe(AudioStems.SfxOrder, 1f);
}
/// &lt;summary&gt;
/// T010-b AD_ORDER_REFRESH clip slot: the boost-refresh tap fires the order
/// chime stem (same "the board changed" family) until a dedicated stem ever
/// ships - the slot itself is the seam, the counter proves the wiring.
/// &lt;/summary&gt;
public static void PlayOrderRefresh()
{
    var host = EnsureHost();
    host.OrderRefreshFired++;
    host.PlayOneShotSafe(AudioStems.SfxOrder, 1f);
}
/// &lt;summary&gt;
/// X1079 harvest stem (census-debt clearance): the petal-pluck cascade
/// fires on the real harvest pay path (GardenView.OnSlotTapped bloom
/// branch, result.Ok). Same silent-pending law as the trio.
/// &lt;/summary&gt;
public static void PlayHarvestHook()
{
```

```
            var host = EnsureHost();
            host.HarvestHookFired++;
            host.PlayOneShotSafe(AudioStems.SfxHarvest, 0.9f);
        }
        /// &lt;summary&gt;
        /// X1079 share stem (census-debt clearance): the warm marimba
        /// confirmation fires on the real copy-tap path (ShareView.OnCopyTapped,
        /// after the code composes). Same silent-pending law as the trio.
        /// &lt;/summary&gt;
        public static void PlayShareHook()
        {
            var host = EnsureHost();
            host.ShareHookFired++;
            host.PlayOneShotSafe(AudioStems.SfxShare, 1f);
        }
        public static void StartBgm()
        {
            var host = EnsureHost();
            host.BgmStartCalls++;
            AudioClip clip = G19AudioRuntime.Get(AudioStems.BgmGarden);
            host.BgmClipPresent = clip != null;
            if (clip == null || !Application.isPlaying) return;
            if (host._bgm == null) return;
            host._bgm.clip = clip;
            host._bgm.Play();
        }
        void PlayOneShotSafe(string stem, float volume)
        {
            AudioClip clip = G19AudioRuntime.Get(stem);
            if (clip == null || !Application.isPlaying) return; // silent pending law
            if (_sfx == null) return;
            _sfx.PlayOneShot(clip, volume);
        }
    }
}
// AudioStems.cs
namespace Sy.BloomHaven.Audio
{
    /// &lt;summary&gt;
    /// Audio stem keys (T008): filename stems of the T001 hook trio + BGM.
    /// Stems follow the fleet naming law (g19_*); the G19AudioSetup editor
    /// tool scans Assets/_Game/Art/Audio and registers clips by these stems.
    /// T001 spec: bloom hook = xylophone+bell arpeggio 0.9s, seed thud =
    /// soft soil thud, order chime = wind-chime gliss, garden BGM loop.
    /// X1079 census-debt clearance: harvest = petal-pluck cascade, share =
    /// warm marimba confirmation chime (the two stem constants close the
    /// item6_census harvest/share channel deficits).
    /// &lt;/summary&gt;
    public static class AudioStems
    {
```

```
        public const string BgmGarden = "g19_bgm_garden";
        public const string SfxBloom = "g19_sfx_bloom";
        public const string SfxPlant = "g19_sfx_plant";
        public const string SfxOrder = "g19_sfx_order";
        public const string SfxHarvest = "g19_sfx_harvest";
        public const string SfxShare = "g19_sfx_share";
        public static readonly string[] All = { BgmGarden, SfxBloom, SfxPlant, SfxOrder, SfxHarvest, SfxShare };
    }
}
// G19AudioCatalog.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.BloomHaven.Audio
{
    /// &lt;summary&gt;
    /// Real-clip audio catalog (T008, G11 WorkerAudioCatalog paradigm): the
    /// G19AudioSetup editor tool scans Assets/_Game/Art/Audio and serializes
    /// AudioClip references here so runtime code resolves sound by filename
    /// stem. The asset lives at Assets/_Game/Resources/G19AudioCatalog.asset
    /// (Resources keeps runtime lookup build-safe).
    /// &lt;/summary&gt;
    [CreateAssetMenu(fileName = "G19AudioCatalog", menuName = "Sy/G19 Audio Catalog")]
    public sealed class G19AudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;     // filename stem, e.g. "g19_sfx_bloom"
            public AudioClip clip;  // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // The serializer may revive the instance with a stale cache; drop it
            // so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
```

```
                }
            }
            return _index.TryGetValue(key, out var c) ? c : null;
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper. Every consumer MUST treat null as "clip missing
    /// -&gt; stay silent" so headless gates stay green on any project state
    /// (catalog absent, audio pending T001 quota, name typos) - the same
    /// null-safe law as ArtRuntime/WorkerAudioRuntime.
    /// &lt;/summary&gt;
    public static class G19AudioRuntime
    {
        static G19AudioCatalog _catalog;
        public static G19AudioCatalog Catalog
        {
            get
            {
                if (_catalog == null) _catalog = Resources.Load&lt;G19AudioCatalog&gt;("G19AudioCatalog");
                return _catalog;
            }
        }
        /// &lt;summary&gt;Force a re-read (editor tests after the setup tool rebuilds the asset).&lt;/summary&gt;
        public static void ResetCache()
        {
            _catalog = null;
        }
        public static AudioClip Get(string key)
        {
            var c = Catalog;
            return c != null ? c.Get(key) : null;
        }
    }
}
// CodexSystem.cs
using System;
using System.Collections.Generic;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Save;
namespace Sy.BloomHaven.Codex
{
    /// &lt;summary&gt;
    /// Floriography codex state machine (M/S5, T010-a data layer). First bloom of a
    /// species unlocks its codex entry (idempotent); every bloom bumps the per-species
    /// bloom count for the codex card. Subscribes to the garden bloom event, so offline
    /// growth (which rides the same monotonic Tick path as live play, T003/T007 law)
    /// unlocks entries too. Rendering lands with the S03 page in T010-b.
    /// &lt;/summary&gt;
    public sealed class CodexSystem
    {
```

```
readonly GardenSystem _garden;
readonly SaveState _save;
public CodexSystem(GardenSystem garden, SaveState save)
{
    _garden = garden ?? throw new ArgumentNullException("garden");
    _save = save ?? throw new ArgumentNullException("save");
    if (_save.Codex == null) _save.Codex = new List&lt;CodexEntry&gt;(); // validator healing
    _garden.OnBloom += HandleBloom;
}
public int UnlockedCount { get { return _save.Codex.Count; } }
/// &lt;summary&gt;Collection complete when every frozen-roster species is unlocked.&lt;/summary&gt;
public bool IsComplete { get { return UnlockedCount &gt;= SpeciesTable.Count; } }
public IReadOnlyList&lt;CodexEntry&gt; Entries { get { return _save.Codex; } }
public bool IsUnlocked(string speciesId)
{
    return Find(speciesId) != null;
}
/// &lt;summary&gt;Blooms recorded for a species (0 while locked, honest face for the card).&lt;/summary&gt;
public int BloomCount(string speciesId)
{
    CodexEntry entry = Find(speciesId);
    return entry == null ? 0 : entry.BloomCount;
}
/// &lt;summary&gt;
/// Record a bloom: the first one unlocks the entry with its timestamp, every
/// bloom bumps the count. Unknown species ids are rejected (frozen roster law).
/// &lt;/summary&gt;
public bool NoteBloom(string speciesId, double nowSeconds)
{
    SpeciesDef def = SpeciesTable.Get(speciesId);
    if (def == null) return false;
    CodexEntry entry = Find(speciesId);
    if (entry == null)
    {
        entry = new CodexEntry { SpeciesId = def.Id, UnlockedAt = nowSeconds, BloomCount = 0 };
        _save.Codex.Add(entry);
    }
    entry.BloomCount++;
    return true;
}
void HandleBloom(BloomBeat beat)
{
    NoteBloom(beat.SpeciesId, _garden.Now);
}
CodexEntry Find(string speciesId)
{
    for (int i = 0; i &lt; _save.Codex.Count; i++)
        if (string.Equals(_save.Codex[i].SpeciesId, speciesId, StringComparison.Ordinal))
            return _save.Codex[i];
    return null;
```

```
        }
    }
}
// CodexView.cs
// G19 codex page host (T010-b): S03_Codex overlay - the collection + boosts
// hub. Opens from the S01 codex entry button. Renders one card per frozen
// species: unlocked cards show the localized name, the floriography + lore
// lines (T010-c content batch) and the live bloom count; locked cards keep
// a silhouette face (no name, no lore leak). The boosts section carries the
// three rewarded placements (U091 pure-optional family): speed up growth /
// double next harvest / refresh the order board. Zero-pressure law: the
// section hides itself the moment the U149 cap has no room (gentle resting
// line, no error face), a tap never spends an ad it cannot use, and the
// whole game completes with zero ad calls. Ad capsules stamp the WALL clock
// so the rolling 24h window survives relaunches. Copy lives behind cfg_i18n
// keys (T008 U114 L0).
using System;
using Sy.BloomHaven.Audio;
using Sy.BloomHaven.Boot;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Iaa;
using Sy.BloomHaven.Localize;
using Sy.BloomHaven.Orders;
using Sy.BloomHaven.Save;
using Sy.BloomHaven.UI;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.BloomHaven.Codex
{
    public sealed class CodexView : MonoBehaviour
    {
        public const string PageRootName = "S03_Codex"; // T006 page-list name
        public const int PageSortOrder = 2;              // modal family: share(2) / codex(2), guide(3) above
        public const float ReferenceWidth = 720f;
        public const float ReferenceHeight = 1280f;
        // element contract (PagesCheck zero-missing law, requirement doc S03 face)
        public const string StatusFaceName = "CodexStatus";
        public const string BoostsRootName = "BoostsRoot";
        public const string RestingFaceName = "BoostsResting";
        public const string SpeedupButtonName = "SpeedupButton";
        public const string DoubleButtonName = "DoubleButton";
        public const string RefreshButtonName = "RefreshButton";
        public const string CloseButtonName = "CloseButton";
        public const string CardPrefix = "CodexCard";
        public const string CardNameSuffix = "Name";
        public const string CardLoreSuffix = "Lore";
        public const string CardBloomSuffix = "Blooms";
        // T011-b reskin contract: the flower codex book page (AR-04) backs
```

```
// the species cards; art missing keeps the modal's dim backdrop.
public const string BookArtName = "CodexBookArt";
public const string BookArtKey = "codex_book";
public const float BookArtSize = 640f;  // square 512x512 spec scaled to the card grid
public const float BookArtY = 90f;      // frames the four cards, below the title
// counters for future play-mode gates (redline11-style assertions)
public int SpeedupTaps;
public int DoubleTaps;
public int RefreshTaps;
public int LastOfferOk = -1; // -1 = no attempt, 0 = rejected, 1 = offered
public bool IsOpen;
public static readonly Color CardBgColor = new Color(0.22f, 0.20f, 0.28f, 0.95f);
public static readonly Color LockedBgColor = new Color(0.13f, 0.12f, 0.16f, 0.95f);
public static readonly Color BloomGoldColor = new Color(1f, 0.85f, 0.45f, 1f);
SaveSystem _facade;
CodexSystem _codex;
IaaSystem _iaa;
GardenSystem _garden;
Text _statusFace;
GameObject _boostsRoot;
Text _restingFace;
GameObject _panel;
bool _built;
int _lastSignature = -1;
readonly Text[] _cardName = new Text[SpeciesTable.Count];
readonly Text[] _cardLore = new Text[SpeciesTable.Count];
readonly Text[] _cardBloom = new Text[SpeciesTable.Count];
readonly Image[] _cardBg = new Image[SpeciesTable.Count];
static bool _subscribed;
/// &lt;summary&gt;
/// Wire the codex page onto S19_Main as soon as it finishes loading
/// (same static-installer family as GardenView). The panel starts
/// hidden - S01 owns the screen until the entry tap.
/// &lt;/summary&gt;
public static void InstallOnSceneLoad()
{
    if (_subscribed) return;
    _subscribed = true;
    SceneManager.sceneLoaded += OnSceneLoaded;
}
static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneIds.Main) return;
    EnsureInstalled();
}
public static CodexView EnsureInstalled()
{
    CodexView view = (CodexView)FindObjectOfType(typeof(CodexView));
    if (view != null) return view;
    // input floor (U184 family lesson): programmatic UGUI taps die
```

```
    // without an EventSystem in the loaded scene.
    if (FindObjectOfType(typeof(EventSystem)) == null)
        new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
    var go = new GameObject(PageRootName, typeof(CodexView));
    return go.GetComponent&lt;CodexView&gt;();
}
// ----- gate seams (pure static faces, PagesCheck/CodexIaaCheck) -----
/// &lt;summary&gt;Boost section precheck: visible only while the U149 cap has room.&lt;/summary&gt;
public static bool BoostsVisible(IaaSystem iaa, double nowWallSeconds)
{
    return iaa != null &amp;&amp; iaa.CapHasRoom(nowWallSeconds);
}
/// &lt;summary&gt;True when at least one slot is mid-grow (speedup precheck face).&lt;/summary&gt;
public static bool HasGrowingSlots(GardenSystem garden)
{
    if (garden == null) return false;
    var slots = garden.State.Slots;
    for (int i = 0; i &lt; slots.Length; i++)
        if (slots[i].Stage == SlotStage.Growing) return true;
    return false;
}
/// &lt;summary&gt;
/// Route a placement type to its mechanism position. Returns false when the
/// effect cannot land (no growing slot / unknown type), so the caller can
/// avoid spending an ad on a no-op.
/// &lt;/summary&gt;
public static bool ApplyBoostEffect(string placementType, GardenSystem garden, OrderSystem orders)
{
    if (string.IsNullOrEmpty(placementType)) return false;
    if (string.Equals(placementType, IaaSystem.AdPlantSpeedup, StringComparison.Ordinal))
        return garden != null &amp;&amp; garden.SpeedupGrowing() &gt; 0;
    if (string.Equals(placementType, IaaSystem.AdHarvestDouble, StringComparison.Ordinal))
    {
        if (garden == null) return false;
        garden.HarvestDoublePending = true;
        return true;
    }
    if (string.Equals(placementType, IaaSystem.AdOrderRefresh, StringComparison.Ordinal))
    {
        if (orders == null) return false;
        orders.RefreshBoard();
        return true;
    }
    return false;
}
/// &lt;summary&gt;Card name face: species name once unlocked, silhouette copy while locked.&lt;/summary&gt;
public static string CardNameFace(CodexSystem codex, SpeciesDef def)
{
    if (def == null) return string.Empty;
    return codex != null &amp;&amp; codex.IsUnlocked(def.Id)
```

```
        ? LocalizeService.Get(def.NameKey)
        : LocalizeService.Get("codex.locked");
}
/// &lt;summary&gt;Card lore face: floriography plus the lore line once unlocked
/// (T010-c content batch), reveal hint while locked (no leak).&lt;/summary&gt;
public static string CardLoreFace(CodexSystem codex, SpeciesDef def)
{
    if (def == null) return string.Empty;
    return codex != null &amp;&amp; codex.IsUnlocked(def.Id)
        ? LocalizeService.Get(def.FloriographyKey) + "\n" + LocalizeService.Get(def.LoreKey)
        : LocalizeService.Get("codex.locked_hint");
}
/// &lt;summary&gt;Card count face: live bloom count once unlocked, empty while locked.&lt;/summary&gt;
public static string CardBloomFace(CodexSystem codex, SpeciesDef def)
{
    if (def == null || codex == null || !codex.IsUnlocked(def.Id)) return string.Empty;
    return LocalizeService.GetF("codex.blooms", codex.BloomCount(def.Id));
}
/// &lt;summary&gt;Unlocked card tint: the species family color dimmed onto the card bg.&lt;/summary&gt;
public static Color CardTint(string speciesId)
{
    Color family = GardenView.ResolveSpeciesColor(speciesId);
    return new Color(family.r * 0.35f + 0.10f, family.g * 0.35f + 0.10f, family.b * 0.35f + 0.12f, 0.95f);
}
// ----- page life -----
void Start()
{
    _facade = SaveHost.Current; // facade owns garden/orders/codex/iaa engines (T010-b wiring)
    _codex = _facade.Codex;
    _iaa = _facade.Iaa;
    _garden = _facade.Garden;
    BuildUi();
    SetOpen(false);
}
/// &lt;summary&gt;Entry toggle from the S01 codex button.&lt;/summary&gt;
public void Toggle()
{
    SetOpen(!IsOpen);
}
void SetOpen(bool open)
{
    IsOpen = open;
    if (_panel == null) return;
    _panel.SetActive(open);
    if (!open) return;
    _lastSignature = -1; // force one refresh on every open
    RefreshCards();
    RefreshBoosts();
    if (_statusFace != null) _statusFace.text = string.Empty;
}
```

```
void Update()
{
    if (!IsOpen || _codex == null) return;
    int signature = CollectionSignature();
    if (signature == _lastSignature) return;
    _lastSignature = signature;
    RefreshCards();
}
int CollectionSignature()
{
    // cheap poll: unlocked count * 1000 + total bloom count (4 species)
    int total = _codex.UnlockedCount * 1000;
    for (int i = 0; i &lt; SpeciesTable.Count; i++)
    {
        SpeciesDef def = SpeciesTable.Get(i);
        if (def == null) continue;
        total += _codex.BloomCount(def.Id);
    }
    return total;
}
// ----- boost taps (U091 pure-optional family, zero-pressure law) -----
void OnSpeedupTapped()
{
    SpeedupTaps++;
    if (!PrecheckCapRoom()) return;
    if (!HasGrowingSlots(_garden))
    {
        SetStatus(LocalizeService.Get("codex.boost_none_growing"));
        return; // never spend an ad on a no-op garden
    }
    if (!OfferNow(IaaSystem.AdPlantSpeedup)) return;
    ApplyBoostEffect(IaaSystem.AdPlantSpeedup, _garden, _facade.Orders);
    SetStatus(LocalizeService.Get("codex.boost_speedup_done"));
    RefreshBoosts();
}
void OnDoubleTapped()
{
    DoubleTaps++;
    if (!PrecheckCapRoom()) return;
    if (!OfferNow(IaaSystem.AdHarvestDouble)) return;
    ApplyBoostEffect(IaaSystem.AdHarvestDouble, _garden, _facade.Orders);
    SetStatus(LocalizeService.Get("codex.boost_double_done"));
    RefreshBoosts();
}
void OnRefreshTapped()
{
    RefreshTaps++;
    if (!PrecheckCapRoom()) return;
    if (!OfferNow(IaaSystem.AdOrderRefresh)) return;
    ApplyBoostEffect(IaaSystem.AdOrderRefresh, _garden, _facade.Orders);
```

```
    AudioHost.PlayOrderRefresh(); // T010-b: refresh clip slot on the real success path
    SetStatus(LocalizeService.Get("codex.boost_refresh_done"));
    RefreshBoosts();
}
/// &lt;summary&gt;Cap precheck: zero-tears toast when the U149 window is full.&lt;/summary&gt;
bool PrecheckCapRoom()
{
    if (BoostsVisible(_iaa, SaveStore.WallSeconds())) return true;
    LastOfferOk = 0;
    SetStatus(LocalizeService.Get("codex.boost_cap_full"));
    return false;
}
/// &lt;summary&gt;Offer through the ledger; a cap-race rejection lands on the same gentle toast.&lt;/summary&gt
bool OfferNow(string placementType)
{
    bool offered = _iaa.OfferAd(placementType, SaveStore.WallSeconds());
    LastOfferOk = offered ? 1 : 0;
    if (!offered) SetStatus(LocalizeService.Get("codex.boost_cap_full"));
    return offered;
}
void SetStatus(string message)
{
    if (_statusFace != null) _statusFace.text = message;
}
void OnCloseTapped()
{
    SetOpen(false);
}
// ----- card + boost rendering -----
void RefreshCards()
{
    for (int i = 0; i &lt; SpeciesTable.Count; i++)
    {
        SpeciesDef def = SpeciesTable.Get(i);
        if (def == null) continue;
        bool unlocked = _codex.IsUnlocked(def.Id);
        if (_cardName[i] != null) _cardName[i].text = CardNameFace(_codex, def);
        if (_cardLore[i] != null) _cardLore[i].text = CardLoreFace(_codex, def);
        if (_cardBloom[i] != null)
        {
            _cardBloom[i].text = CardBloomFace(_codex, def);
            _cardBloom[i].color = unlocked ? BloomGoldColor : new Color(1f, 1f, 1f, 0.3f);
        }
        if (_cardBg[i] != null) _cardBg[i].color = unlocked ? CardTint(def.Id) : LockedBgColor;
    }
}
void RefreshBoosts()
{
    bool visible = BoostsVisible(_iaa, SaveStore.WallSeconds());
    if (_boostsRoot != null) _boostsRoot.SetActive(visible);
```

```
    if (_restingFace != null) _restingFace.gameObject.SetActive(!visible);
}
// ----- UI construction (programmatic, zero scene authoring) -----
void BuildUi()
{
    if (_built) return;
    _built = true;
    var canvasGo = new GameObject("CodexCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    canvasGo.transform.SetParent(transform, false); // page root owns the canvas
    var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = PageSortOrder; // overlay law: garden 0 -&gt; board 1 -&gt; codex/share 2
    var scaler = canvasGo.GetComponent&lt;CanvasScaler&gt;();
    // U184: ScaleWithScreenSize or the reference frame never drives
    // device scale (raw ConstantPixelSize = shrunken UI on phone).
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(ReferenceWidth, ReferenceHeight);
    _panel = new GameObject("CodexPanel", typeof(RectTransform), typeof(Image));
    _panel.transform.SetParent(canvasGo.transform, false);
    var panel = (RectTransform)_panel.transform;
    panel.anchorMin = Vector2.zero;
    panel.anchorMax = Vector2.one;
    panel.sizeDelta = Vector2.zero; // full-screen dim backdrop
    panel.anchoredPosition = Vector2.zero;
    var backdrop = _panel.GetComponent&lt;Image&gt;();
    backdrop.color = new Color(0.08f, 0.06f, 0.10f, 0.82f);
    backdrop.raycastTarget = true; // modal: swallow taps behind the panel
    // T011-b: the codex book page (AR-04) backs the species cards,
    // null-safe - art missing keeps the dim-backdrop face.
    Sy.BloomHaven.UI.PageArt.BuildNode(_panel.transform, BookArtName, 0f, BookArtY,
        BookArtSize, BookArtSize, Sy.Core.ArtRuntime.Get(BookArtKey));
    var title = MakeText(_panel.transform, "CodexTitle", 32f, new Vector2(0f, 430f), 680, 44);
    title.text = LocalizeService.Get("codex.title");
    for (int i = 0; i &lt; SpeciesTable.Count; i++) BuildCard(_panel.transform, i);
    var boostsTitle = MakeText(_panel.transform, "BoostsTitle", 18f, new Vector2(0f, -236f), 680, 28);
    boostsTitle.color = new Color(1f, 1f, 1f, 0.75f);
    boostsTitle.text = LocalizeService.Get("codex.boosts_title");
    _boostsRoot = new GameObject(BoostsRootName, typeof(RectTransform));
    _boostsRoot.transform.SetParent(_panel.transform, false);
    var boostsRt = (RectTransform)_boostsRoot.transform;
    boostsRt.anchorMin = new Vector2(0.5f, 0.5f);
    boostsRt.anchorMax = new Vector2(0.5f, 0.5f);
    boostsRt.sizeDelta = new Vector2(ReferenceWidth, 200f);
    boostsRt.anchoredPosition = Vector2.zero;
    var speedup = MakeButton(_boostsRoot.transform, SpeedupButtonName,
        LocalizeService.Get("codex.boost_speedup"), -215f, -290f, 205f, 52f);
    speedup.image.color = new Color(0.36f, 0.52f, 0.42f, 1f);
    speedup.onClick.AddListener(OnSpeedupTapped);
    var doubleUp = MakeButton(_boostsRoot.transform, DoubleButtonName,
```

```
            LocalizeService.Get("codex.boost_double"), 0f, -290f, 205f, 52f);
        doubleUp.image.color = new Color(0.42f, 0.56f, 0.86f, 1f);
        doubleUp.onClick.AddListener(OnDoubleTapped);
        var refresh = MakeButton(_boostsRoot.transform, RefreshButtonName,
            LocalizeService.Get("codex.boost_refresh"), 215f, -290f, 205f, 52f);
        refresh.image.color = new Color(0.80f, 0.62f, 0.36f, 1f);
        refresh.onClick.AddListener(OnRefreshTapped);
        // zero-tears resting line: shows exactly when the boosts section hides
        _restingFace = MakeText(_panel.transform, RestingFaceName, 18f, new Vector2(0f, -290f), 640, 32);
        _restingFace.color = new Color(1f, 0.85f, 0.45f, 0.85f);
        _restingFace.text = LocalizeService.Get("codex.boosts_resting");
        _statusFace = MakeText(_panel.transform, StatusFaceName, 18f, new Vector2(0f, -368f), 660, 30);
        _statusFace.color = new Color(1f, 0.85f, 0.45f, 1f);
        _statusFace.text = string.Empty;
        var close = MakeButton(_panel.transform, CloseButtonName,
            LocalizeService.Get("codex.close"), 0f, -500f, 140f, 46f);
        close.image.color = new Color(0.42f, 0.36f, 0.36f, 1f);
        close.onClick.AddListener(OnCloseTapped);
    }
    void BuildCard(Transform parent, int i)
    {
        float x = (i % 2 == 0) ? -175f : 175f;
        float y = (i &lt; 2) ? 190f : -40f;
        var cardGo = new GameObject(CardPrefix + i, typeof(RectTransform), typeof(Image));
        cardGo.transform.SetParent(parent, false);
        var rt = (RectTransform)cardGo.transform;
        rt.sizeDelta = new Vector2(330f, 220f);
        rt.anchoredPosition = new Vector2(x, y);
        _cardBg[i] = cardGo.GetComponent&lt;Image&gt;();
        _cardBg[i].color = LockedBgColor;
        _cardBg[i].raycastTarget = false;
        _cardName[i] = MakeText(cardGo.transform, CardNameSuffix + i, 22f, new Vector2(0f, 78f), 310, 32);
        _cardLore[i] = MakeText(cardGo.transform, CardLoreSuffix + i, 16f, new Vector2(0f, 6f), 300, 100);
        _cardLore[i].horizontalOverflow = HorizontalWrapMode.Wrap;
        _cardLore[i].color = new Color(1f, 1f, 1f, 0.9f);
        _cardBloom[i] = MakeText(cardGo.transform, CardBloomSuffix + i, 18f, new Vector2(0f, -85f), 300, 28);
    }
    static Button MakeButton(Transform parent, string name, string label,
        float x, float y, float w, float h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = new Vector2(x, y);
        var button = go.GetComponent&lt;Button&gt;();
        button.targetGraphic = go.GetComponent&lt;Image&gt;(); // U184: real target graphic
        button.transition = Selectable.Transition.ColorTint;
        var text = MakeText(go.transform, "Label", 15f, Vector2.zero, (int)w - 10, (int)h - 8);
        text.text = label;
```

```
            return button;
        }
        static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.sizeDelta = new Vector2(w, h);
            rt.anchoredPosition = pos;
            var t = go.GetComponent&lt;Text&gt;();
            t.font = HudFont.Resolve();
            t.fontSize = Mathf.RoundToInt(size);
            t.alignment = TextAnchor.MiddleCenter;
            t.color = Color.white;
            t.raycastTarget = false; // never steal a button tap
            return t;
        }
    }
}
// GardenModel.cs
using System;
namespace Sy.BloomHaven.Garden
{
    /// &lt;summary&gt;Growth lifecycle of a slot (cozy law: no wither, no death state).&lt;/summary&gt;
    public enum SlotStage
    {
        Empty,
        Growing,
        Bloom
    }
    /// &lt;summary&gt;Per-slot persistent state ({species, plantedAt, bloomAt, stage, renovated} — save.slots face).&lt
    [Serializable]
    public sealed class SlotState
    {
        public string SpeciesId = string.Empty;
        public double PlantedAt;
        /// &lt;summary&gt;Session-clock seconds when the slot bloomed (T012-g c1): the
        /// keep-window math reads (Now - BloomedAt) vs BloomKeepSec; Resume
        /// re-stamps in-bloom slots so the away gap never advances wilt (cozy law).&lt;/summary&gt;
        public double BloomedAt;
        public SlotStage Stage = SlotStage.Empty;
        /// &lt;summary&gt;Renovation ground face: order completion flips the owning block's slots (M4).&lt;/summary&gt;
        public bool Renovated;
    }
    /// &lt;summary&gt;Whole-garden persistent state (save.slots + save.wallet faces).&lt;/summary&gt;
    [Serializable]
    public sealed class GardenState
    {
        public SlotState[] Slots = new SlotState[GardenSystem.SlotCount];
        public int Petals;
```

```
    public string SelectedSpeciesId = SpeciesTable.Bluebell;
    public GardenState()
    {
        for (int i = 0; i &lt; Slots.Length; i++) Slots[i] = new SlotState();
    }
}
/// &lt;summary&gt;
/// Bloom-instant payload for the three-channel beat (M2): tint + caption + flash
/// fire together on one beat; renderers subscribe to GardenSystem.OnBloom.
/// &lt;/summary&gt;
public readonly struct BloomBeat
{
    public readonly int SlotIndex;
    public readonly string SpeciesId;
    public readonly bool TintChannel;
    public readonly bool CaptionChannel;
    public readonly bool FlashChannel;
    public readonly double BeatSeconds;
    public BloomBeat(int slotIndex, string speciesId, double beatSeconds)
    {
        SlotIndex = slotIndex;
        SpeciesId = speciesId;
        TintChannel = true;
        CaptionChannel = true;
        FlashChannel = true;
        BeatSeconds = beatSeconds;
    }
}
/// &lt;summary&gt;
/// Wilt payload (T012-g c1 bloom keep-window): fired once per slot when a
/// standing bloom reaches the end of its keep window - the slot returns to
/// barren (Empty, no resource loss, cozy law) and the view plays the soft
/// fade. Renderers subscribe to GardenSystem.OnWilt.
/// &lt;/summary&gt;
public readonly struct WiltBeat
{
    public readonly int SlotIndex;
    public readonly string SpeciesId;
    public WiltBeat(int slotIndex, string speciesId)
    {
        SlotIndex = slotIndex;
        SpeciesId = speciesId;
    }
}
/// &lt;summary&gt;Harvest outcome; Gained is the amount actually credited (cap clips it).&lt;/summary&gt;
public readonly struct HarvestResult
{
    public readonly bool Ok;
    public readonly int Gained;
    public readonly int WalletAfter;
```

```
        public HarvestResult(bool ok, int gained, int walletAfter)
        {
            Ok = ok;
            Gained = gained;
            WalletAfter = walletAfter;
        }
    }
}
// GardenSystem.cs
using System;
namespace Sy.BloomHaven.Garden
{
    /// &lt;summary&gt;
    /// Plant/grow/harvest rules engine (M1/M2/M3 + T012-g c1 bloom keep-window).
    /// Pure C# with an injected clock: Tick(nowSeconds) both advances live play
    /// and replays offline growth, because stage math is always (now - plantedAt)
    /// vs growSeconds (T007 reuses it as-is). The keep-window (bloom wilt) rides
    /// the SAME session clock: a bloom wilts at BloomedAt + BloomKeepSec (soft
    /// fade, slot returns to barren, zero resource loss - cozy law); the away
    /// gap never advances it because SaveStore.Resume re-stamps in-bloom slots.
    /// &lt;/summary&gt;
    public sealed class GardenSystem
    {
        public const int SlotCount = 16;             // M1: preset slots, fixed count
        public const double BloomBeatSeconds = 0.3;  // M2: three-channel beat rhythm
        public const int WalletCap = 999;            // M3: petals wallet cap
        // T012-g c1 (X1291 design freeze): bloom keep-window anchor B - the
        // frozen band is [2.0, 6.0], first anchor B=3.0 (~10.5s on the 3.5s
        // family). X1293 single calibration wave (X1086 single-shot tuning
        // law analog) picks the tight in-band edge B=2.0 (bluebell keep 6.0s):
        // the B=3.0 matrix measured strategy_rate 0.963 &gt; the 0.95 F1 ceiling
        // with close-call wins at 13% - shortening the keep raises mid-pursuit
        // wilts (later completions, more losses) which is the only frozen
        // lever that moves both faces toward the band. Out-of-band after this
        // wave files as honest RED, never a gate edit (U098).
        public const double WiltB = 2.0;
        public GardenState State { get; }
        public double Now { get; private set; }
        /// &lt;summary&gt;
        /// T010-b AD_HARVEST_DOUBLE mechanism face: one-shot transient flag - the
        /// next successful harvest pays double (still clipped at the wallet cap),
        /// then the boost is spent. Deliberately NOT persisted: a relaunch never
        /// keeps a paid-but-unused double waiting (honest face, save-stable).
        /// &lt;/summary&gt;
        public bool HarvestDoublePending { get; set; }
        /// &lt;summary&gt;Raised once per slot when it crosses its grow threshold.&lt;/summary&gt;
        public event Action&lt;BloomBeat&gt; OnBloom;
        /// &lt;summary&gt;Raised once per slot when a standing bloom reaches the end of
        /// its keep window (T012-g c1): the slot returns to barren, the view plays
        /// the soft fade. Zero resource loss, no negative popup (cozy law).&lt;/summary&gt;
```

```
public event Action&lt;WiltBeat&gt; OnWilt;
/// &lt;summary&gt;Bloom keep-window seconds for a species family (T012-g c1):
/// GrowSeconds * WiltB - the freeze mirrors the gate sitting judge's
/// family math, so measured and enforced windows stay in lockstep.&lt;/summary&gt;
public static double BloomKeepSec(SpeciesDef def)
    =&gt; def == null ? 0.0 : def.GrowSeconds * WiltB;
public GardenSystem(GardenState state = null)
{
    State = state ?? new GardenState();
    if (State.Slots == null || State.Slots.Length != SlotCount)
        throw new ArgumentException("garden state must carry exactly " + SlotCount + " slots");
}
/// &lt;summary&gt;Advance world time; each Growing slot that reaches its grow window
/// blooms (stamping the keep-window clock), and each standing bloom whose keep
/// window closes wilts back to barren (independent per slot, same session clock).&lt;/summary&gt;
public void Tick(double nowSeconds)
{
    if (nowSeconds &lt; Now) return; // monotonic clock guard
    Now = nowSeconds;
    var slots = State.Slots;
    for (int i = 0; i &lt; slots.Length; i++)
    {
        if (slots[i].Stage == SlotStage.Growing)
        {
            SpeciesDef def = SpeciesTable.Get(slots[i].SpeciesId);
            if (def == null) continue;
            if (nowSeconds - slots[i].PlantedAt &gt;= def.GrowSeconds)
            {
                slots[i].Stage = SlotStage.Bloom;
                slots[i].BloomedAt = nowSeconds; // keep-window stamp (T012-g c1)
                OnBloom?.Invoke(new BloomBeat(i, slots[i].SpeciesId, BloomBeatSeconds));
            }
        }
        else if (slots[i].Stage == SlotStage.Bloom)
        {
            WiltSlotIfDue(i);
        }
    }
}
/// &lt;summary&gt;
/// Explicit wilt sweep (T012-g c1): the same keep-window math Tick
/// enforces, for board-entry callers (OrderSystem.EnsureBoard and
/// RefreshBoard mirror the X1290 SweepExpired pattern) so the board a
/// player interacts with never judges against stale stock. Returns the
/// number of slots wilted by this call.
/// &lt;/summary&gt;
public int SweepWilt()
{
    int wilted = 0;
    var slots = State.Slots;
```

```
    for (int i = 0; i &lt; slots.Length; i++)
    {
        if (slots[i].Stage != SlotStage.Bloom) continue;
        if (WiltSlotIfDue(i)) wilted++;
    }
    return wilted;
}
/// &lt;summary&gt;Wilt one slot if its keep window closed: the flower returns to
/// barren (slot Empty, no payout, no loss booking) and OnWilt fires for the
/// soft fade. A missing species def never wilts (null-safe guard).&lt;/summary&gt;
bool WiltSlotIfDue(int index)
{
    SlotState slot = State.Slots[index];
    SpeciesDef def = SpeciesTable.Get(slot.SpeciesId);
    if (def == null) return false;
    if (Now - slot.BloomedAt &lt; BloomKeepSec(def)) return false;
    string speciesId = slot.SpeciesId;
    slot.SpeciesId = string.Empty;
    slot.PlantedAt = 0;
    slot.BloomedAt = 0;
    slot.Stage = SlotStage.Empty;
    OnWilt?.Invoke(new WiltBeat(index, speciesId));
    return true;
}
/// &lt;summary&gt;M1 sole planting entry point. Out-of-range indices and non-empty slots are rejected without p
public bool Plant(int presetSlotIndex)
{
    if (presetSlotIndex &lt; 0 || presetSlotIndex &gt;= SlotCount) return false;
    SlotState slot = State.Slots[presetSlotIndex];
    if (slot.Stage != SlotStage.Empty) return false;
    SpeciesDef def = SpeciesTable.Get(State.SelectedSpeciesId);
    if (def == null) return false;
    slot.SpeciesId = def.Id;
    slot.PlantedAt = Now;
    slot.Stage = SlotStage.Growing;
    return true;
}
/// &lt;summary&gt;M3 harvest: only a bloomed slot pays out; the slot returns to empty; wallet clips at cap.&lt;
public HarvestResult Harvest(int presetSlotIndex)
{
    if (presetSlotIndex &lt; 0 || presetSlotIndex &gt;= SlotCount)
        return new HarvestResult(false, 0, State.Petals);
    SlotState slot = State.Slots[presetSlotIndex];
    if (slot.Stage != SlotStage.Bloom)
        return new HarvestResult(false, 0, State.Petals);
    SpeciesDef def = SpeciesTable.Get(slot.SpeciesId);
    int nominal = def == null ? 0 : def.PetalsOnHarvest;
    if (HarvestDoublePending)
    {
        nominal *= 2; // T010-b: the rewarded double boost doubles the payout base
```

```
                HarvestDoublePending = false; // one-shot: spent on the first successful harvest
            }
            int credited = Math.Min(WalletCap - State.Petals, nominal);
            if (credited &lt; 0) credited = 0;
            State.Petals += credited;
            slot.SpeciesId = string.Empty;
            slot.PlantedAt = 0;
            slot.BloomedAt = 0;
            slot.Stage = SlotStage.Empty;
            return new HarvestResult(true, credited, State.Petals);
        }
        /// &lt;summary&gt;Remaining seconds for a growing slot; -1 when not growing or out of range.&lt;/summary&gt;
        public double SecondsUntilBloom(int presetSlotIndex)
        {
            if (presetSlotIndex &lt; 0 || presetSlotIndex &gt;= SlotCount) return -1;
            SlotState slot = State.Slots[presetSlotIndex];
            if (slot.Stage != SlotStage.Growing) return -1;
            SpeciesDef def = SpeciesTable.Get(slot.SpeciesId);
            if (def == null) return -1;
            double remain = def.GrowSeconds - (Now - slot.PlantedAt);
            return remain &gt; 0 ? remain : 0;
        }
        /// &lt;summary&gt;
        /// T010-b AD_PLANT_SPEEDUP mechanism face: halve the remaining grow window
        /// of every growing slot by rewinding maturity (E4 probe anchor: a fresh
        /// bluebell reaches bloom at 1.50s instead of 3.0s). Same stage math as
        /// offline resume - PlantedAt moves, no new path. Returns slots helped so
        /// callers never spend an ad on an empty garden.
        /// &lt;/summary&gt;
        public int SpeedupGrowing()
        {
            int helped = 0;
            var slots = State.Slots;
            for (int i = 0; i &lt; slots.Length; i++)
            {
                if (slots[i].Stage != SlotStage.Growing) continue;
                SpeciesDef def = SpeciesTable.Get(slots[i].SpeciesId);
                if (def == null) continue;
                double grown = Now - slots[i].PlantedAt;
                if (grown &lt; 0) grown = 0;
                double remain = def.GrowSeconds - grown;
                if (remain &lt;= 0) continue; // at the window edge: the next Tick blooms it anyway
                slots[i].PlantedAt = Now - (grown + remain * 0.5);
                helped++;
            }
            return helped;
        }
    }
}
// GardenView.cs
```

```
// G19 garden page host (T003 slice B): renders the 16-slot garden onto
// S19_Main with zero scene authoring (X027 programmatic UGUI). It owns the
// GardenSystem rules engine, ticks it on the monotonic realtime clock (the
// same math T007 replays offline growth), and subscribes to OnBloom to
// drive the three-channel beat - tint, caption and flash fire together on
// one 0.3s beat, independent per slot. Taps plant an empty patch (selected
// species) or harvest a bloom; payouts float up "+N" and the wallet HUD
// updates. The guide opening beat (T006 second-by-second table: first
// bloom T+0-3s) is seeded by a pre-arrived sprout in slot 0 on a fresh
// garden. Copy lives behind cfg_i18n keys (T008 U114 L0); values stay
// English on the shipped G19Main pixel face (X1302 HUD font chain; a CJK
// arrival needs a census+subset re-cast, U184 risk noted). The T008 hook
// trio fires here: bloom chime on the beat, seed
// thud on plant, BGM loop on page start. The S01 slot faces consume the
// landed AR-02 dual-state tile (ground) and the AR-01 bluebell (bloom
// face) through the ArtRuntime null-safe seam - art missing keeps the
// procedural color-block faces (T012-d U088 re-eval trigger).
using System;
using System.Collections.Generic;
using Sy.BloomHaven.Audio;
using Sy.BloomHaven.Boot;
using Sy.BloomHaven.Codex;
using Sy.BloomHaven.Localize;
using Sy.BloomHaven.Orders;
using Sy.BloomHaven.Save;
using Sy.BloomHaven.Share;
using Sy.BloomHaven.UI;
using Sy.Core;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.BloomHaven.Garden
{
    public sealed class GardenView : MonoBehaviour
    {
        public const string PageRootName = "S01_Garden"; // T006 page-list name
        public const string ShareButtonName = "ShareButton"; // S01 share entry (T006)
        public const string CodexButtonName = "CodexButton"; // S01 codex entry (T010-b)
        public const float ReferenceWidth = 720f;        // portrait fleet frame (G16 proof)
        public const float ReferenceHeight = 1280f;
        public const float CellSize = 150f;
        public const float CellGap = 14f;
        public const float GridAnchorY = -70f;
        public const float PayoutSeconds = 0.7f;         // harvest float feedback
        public const float WiltFadeSeconds = 0.8f;      // T012-g c1: bloom soft-fade life (cozy law, no harsh X)
        public const float CutBeatSeconds = 0.5f;       // S2 staging depth: bouquet goodbye fade life
        public const float SaveIntervalSeconds = 10f;     // T007 slice b: periodic persist cadence
        public const float WelcomeSeconds = 2.6f;         // T007 slice b: offline-accrual bubble life
        public const string SoilBarrenKey = "garden_slot_grid";       // AR-02 wasteland tile (S01 slot ground)
```

```
public const string SoilBloomKey = "garden_slot_grid_bloom";  // AR-02 renovated flower-sea tile
public const string BluebellArtKey = "flower_bluebell";       // AR-01 bloom face (roster SP01)
public const string SproutArtKey = "garden_sprout";          // AR-07 growing face (slot sprout)
public const string BackdropArtKey = "garden_backdrop";      // AR-08 full-frame garden backdrop (S01 canvas gro
public const string BackdropName = "Backdrop";               // AR-08 canvas child (below SafeArea)
public const string ShareChromeKey = "hud_btn_share";        // AR-09 HUD chrome (S01 share entry)
public const string CodexChromeKey = "hud_btn_codex";        // AR-09 HUD chrome (S01 codex entry)
// counters for future play-mode gates (redline11-style assertions)
public int BloomBeatsSeen;
public int PlantTaps;
public int HarvestTaps;
public int WiltFadesSeen;                        // T012-g c1: soft-fade beats seen by gates
public int CutBeatsSeen;                         // S2 staging depth: bouquet-cut beats seen by gates
public int LastCutCount;                         // S2 staging depth: last delivery's cut slot count
public int LastBloomSlot = -1;
public int LastHarvestGained;
public int LastWelcomeAccrued;                    // T007 slice b: bubble face seen by gates
GardenSystem _system;
Text _walletFace;
double _nextSaveAt;
readonly List&lt;SlotFace&gt; _faces = new List&lt;SlotFace&gt;(GardenSystem.SlotCount);
readonly List&lt;PayoutFace&gt; _payouts = new List&lt;PayoutFace&gt;(4);
readonly List&lt;WelcomeFace&gt; _welcome = new List&lt;WelcomeFace&gt;(1);
readonly float[] _flipRemaining = new float[GardenSystem.SlotCount]; // block-flip beat per slot
readonly float[] _wiltRemaining = new float[GardenSystem.SlotCount]; // T012-g c1 soft fade per slot
readonly string[] _wiltSpecies = new string[GardenSystem.SlotCount]; // fading bloom art key per slot
readonly float[] _cutRemaining = new float[GardenSystem.SlotCount];  // S2 bouquet-cut beat per slot
readonly string[] _cutSpecies = new string[GardenSystem.SlotCount];  // cut bloom art key per slot
Button _shareEntryButton; // S01 share entry, hidden while the share modal owns the screen
Text _shareEntryLabel;    // X1614: relabeled to the return action while a visit tour is live
// U088 X1062 float guard: while the share modal is open the garden
// keeps simulating, but its floating labels (payouts, welcome bubble,
// slot captions) stay hidden so nothing peeks from behind the card.
public bool FloatsSuppressed { get; private set; }
public GardenSystem System =&gt; _system;
static bool _subscribed;
sealed class SlotFace
{
    public Image Plant;
    public Image Flash;
    public Image Soil;
    public Text Caption;
    public Button Button;
    public float BeatRemaining;
    public Sprite SoilBarren;   // S01 slot-face art (null-safe seam)
    public Sprite SoilBloom;
    public bool SoilArtLive;
}
sealed class PayoutFace
{
```

```
    public Text Face;
    public float Remaining;
    public float BaseX;
    public float BaseY;
}
sealed class WelcomeFace
{
    public Text Face;
    public float Remaining;
}
void Start() =&gt; Initialize();
/// &lt;summary&gt;
/// Headless boot seam (U085 slice-B patrol): the full Start() face -
/// facade wiring, opening-beat seed, UI build, wallet, welcome bubble,
/// BGM - callable from editor gates where the MonoBehaviour lifecycle
/// never runs. Same code path as live play, zero copy.
/// &lt;/summary&gt;
public void Initialize()
{
    _system = SaveHost.Current.Garden; // T007 slice b: the save facade owns the engine (zero copy-drift)
    _system.OnBloom += HandleBloom;
    _system.OnWilt += HandleWilt; // T012-g c1: soft fade on the wilt beat
    double now = Time.realtimeSinceStartupAsDouble;
    _system.Tick(now); // establish the monotonic clock before seeding
    if (IsFreshGarden(_system.State))
        SeedOpeningBeat(now);
    BuildUi();
    RefreshWallet();
    ShowWelcomeBackBubble(); // T007 slice b: S01 offline-accrual bubble on revisit
    AudioHost.StartBgm(); // T008: garden BGM loop (silent until the T001 clip lands)
    _nextSaveAt = now + SaveIntervalSeconds;
}
void OnDestroy()
{
    if (_system != null)
    {
        _system.OnBloom -= HandleBloom;
        _system.OnWilt -= HandleWilt;
    }
}
/// &lt;summary&gt;
/// Wire the garden page onto S19_Main as soon as it finishes loading.
/// The scene-loaded handler is static because the boot-scene host is
/// destroyed by the single-mode switch before its coroutine resumes.
/// &lt;/summary&gt;
public static void InstallOnSceneLoad()
{
    if (_subscribed) return;
    _subscribed = true;
    SceneManager.sceneLoaded += OnSceneLoaded;
```

```
}
static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneIds.Main) return;
    EnsureInstalled();
}
public static GardenView EnsureInstalled()
{
    GardenView view = (GardenView)FindObjectOfType(typeof(GardenView));
    if (view != null) return view;
    // input floor (U184 family lesson): programmatic UGUI taps die
    // without an EventSystem in the loaded scene.
    if (FindObjectOfType(typeof(EventSystem)) == null)
        new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
    var go = new GameObject(PageRootName, typeof(GardenView));
    return go.GetComponent&lt;GardenView&gt;();
}
/// &lt;summary&gt;A garden is fresh when nothing was ever planted and no petals exist.&lt;/summary&gt;
public static bool IsFreshGarden(GardenState state)
{
    if (state.Petals != 0) return false;
    for (int i = 0; i &lt; state.Slots.Length; i++)
        if (state.Slots[i].Stage != SlotStage.Empty) return false;
    return true;
}
// Guide opening beat (T006 table: first bloom T+0-3s): slot 0 carries
// a sprout that "arrived with you" - plantedAt is wound back so the
// bloom beat lands ~1.5s after the page opens. Same monotonic math
// as offline continuation, no special path.
void SeedOpeningBeat(double now)
{
    SpeciesDef def = SpeciesTable.Get(_system.State.SelectedSpeciesId);
    if (def == null) return;
    SlotState slot = _system.State.Slots[0];
    slot.SpeciesId = def.Id;
    slot.PlantedAt = now - (def.GrowSeconds - 1.5);
    slot.Stage = SlotStage.Growing;
}
void Update() =&gt; DriveFrame(Time.unscaledDeltaTime);
/// &lt;summary&gt;
/// One driven frame (U085 slice-B patrol seam): the full Update() face
/// with an injected beat dt, so editor gates can pump the live view
/// without a running play loop. The garden clock stays the monotonic
/// realtime face - the same math live play grows on.
/// &lt;/summary&gt;
public void DriveFrame(double dt)
{
    double nowUptime = Time.realtimeSinceStartupAsDouble;
    _system.Tick(nowUptime);
    DriveBeats((float)dt);
```

```
    DrivePayouts((float)dt);
    DriveWelcome((float)dt);
    RefreshSlots();
    DriveCutBeats((float)dt); // S2 staging: cut goodbye owns the face first, the staged flip lands on top
    DriveFlipBeats((float)dt); // after RefreshSlots: beat owns caption/flash frames
    DriveWiltFades((float)dt); // T012-g c1: after RefreshSlots the fade owns the fading bloom face
    if (nowUptime &gt;= _nextSaveAt) // T007 slice b: periodic persist, crash loss &lt;= interval
    {
        // X1614 tour guard: while visiting, the live face IS the
        // friend's garden - persisting here would write the friend's
        // face over the player's save. Home-only persist.
        if (!IsVisiting) SaveHost.SaveNow(nowUptime);
        _nextSaveAt = nowUptime + SaveIntervalSeconds;
    }
}
// mobile kill path: pause may be the last callback the process ever gets
void OnApplicationPause(bool pause)
{
    if (pause &amp;&amp; !IsVisiting) SaveHost.SaveNow(Time.realtimeSinceStartupAsDouble); // X1614 tour guard
}
void OnApplicationQuit()
{
    if (!IsVisiting) SaveHost.SaveNow(Time.realtimeSinceStartupAsDouble); // X1614 tour guard
}
// ----- block-flip beat (M4 slice b): renovation gold flash + caption -----
/// &lt;summary&gt;
/// Fire the renovation beat on a block's slots (board page calls this after
/// CompleteOrder flips the block ground). Runs after RefreshSlots in Update
/// so the beat owns the caption/flash channels while it lasts.
/// &lt;/summary&gt;
public void PlayBlockFlipBeat(int[] slotIndexes)
{
    if (slotIndexes == null) return;
    for (int i = 0; i &lt; slotIndexes.Length; i++)
    {
        int index = slotIndexes[i];
        if (index &gt;= 0 &amp;&amp; index &lt; _flipRemaining.Length)
            _flipRemaining[index] = FlipBeatSeconds;
    }
}
void DriveFlipBeats(float dt)
{
    for (int i = 0; i &lt; _flipRemaining.Length; i++)
    {
        if (_flipRemaining[i] &lt;= 0f) continue;
        _flipRemaining[i] -= dt;
        if (i &gt;= _faces.Count) continue;
        SlotFace face = _faces[i];
        float p = 1f - Mathf.Clamp01(_flipRemaining[i] / FlipBeatSeconds);
        if (face.Flash != null)
```

```
            face.Flash.color = new Color(1f, 0.92f, 0.55f, 0.9f * (1f - p)); // renovation gold
        if (face.Caption != null &amp;amp;&amp;amp; !FloatsSuppressed)
        {
            face.Caption.text = LocalizeService.Get("garden.renovated");
            face.Caption.color = new Color(1f, 0.92f, 0.55f, 1f - p);
        }
    }
}
// ----- bouquet-cut beat (S2 order-delivery staging depth) -----
/// &lt;summary&gt;
/// Fire the bouquet goodbye beat on the slots the satisfying tap
/// consumed (order board calls this right after CompleteOrder; the
/// payload comes from the OrderResult - slot index plus the species
/// that stood there, so the fade shows the leaving bloom, not bare
/// ground). Cozy gold ease-out, bouquet caption; the staged block
/// flip that follows owns the terminal renovation caption on overlap.
/// &lt;/summary&gt;
public void PlayBouquetCutBeat(BouquetCut[] cut)
{
    if (cut == null) return;
    CutBeatsSeen++;
    LastCutCount = cut.Length;
    for (int i = 0; i &lt; cut.Length; i++)
    {
        int index = cut[i].SlotIndex;
        if (index &lt; 0 || index &gt;= _cutRemaining.Length) continue;
        _cutRemaining[index] = CutBeatSeconds;
        _cutSpecies[index] = cut[i].SpeciesId ?? string.Empty;
    }
}
/// &lt;summary&gt;
/// Drive the per-slot bouquet-cut fade after RefreshSlots: the slot
/// state is already empty (CompleteOrder cut it), so the beat owns the
/// leaving bloom face for CutBeatSeconds - the species art eases out
/// in delivery gold with a "Bouquet!" caption (art missing = tint-only
/// procedural face, X027 null-safe law). A re-plant into the same slot
/// cancels the fade (the fresh sprout wins).
/// &lt;/summary&gt;
void DriveCutBeats(float dt)
{
    if (_system == null) return;
    for (int i = 0; i &lt; _cutRemaining.Length; i++)
    {
        if (_cutRemaining[i] &lt;= 0f) continue;
        _cutRemaining[i] -= dt;
        if (i &gt;= _faces.Count) continue;
        if (_system.State.Slots[i].Stage != SlotStage.Empty)
        {
            _cutRemaining[i] = 0f; // re-planted mid-fade: the new face wins
            continue;
```

```
{
    OrderTarget target = def.Target;
    switch (target.Kind)
    {
        case OrderTargetKind.ColorFamily:
            return LocalizeService.GetF("order.target.color", target.MinSlots,
                LocalizeService.Get("color." + target.Primary));
        case OrderTargetKind.Species:
            SpeciesDef species = SpeciesTable.Get(target.Primary);
            return LocalizeService.GetF("order.target.species", target.MinSlots,
                species == null ? target.Primary : LocalizeService.Get(species.NameKey));
        case OrderTargetKind.DualColor:
            return LocalizeService.GetF("order.target.dual",
                LocalizeService.Get("color." + target.Primary),
                LocalizeService.Get("color." + target.Secondary), target.MinSlots);
        default:
            return string.Empty;
    }
}
// ----- payout float (harvest-family feedback) -----
void SpawnPayout(float cardX, int gained)
{
    if (_safeRoot == null || _payouts.Count &gt;= 4) return;
    var go = new GameObject("OrderPayout", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(_safeRoot, false);
    var rt = (RectTransform)go.transform;
    rt.anchoredPosition = new Vector2(cardX, CardRowY + 130f);
    var t = go.GetComponent&lt;Text&gt;();
    t.font = HudFont.Resolve();
    t.fontSize = 28;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = ReadyColor;
    t.raycastTarget = false;
    rt.sizeDelta = new Vector2(200f, 40f);
    t.text = LocalizeService.GetF("garden.payout", gained);
    _payouts.Add(new PayoutFace
    {
        Face = t,
        Remaining = PayoutSeconds,
        BaseX = rt.anchoredPosition.x,
        BaseY = rt.anchoredPosition.y
    });
}
void DrivePayouts(float dt)
{
    for (int i = _payouts.Count - 1; i &gt;= 0; i--)
    {
        PayoutFace payout = _payouts[i];
        payout.Remaining -= dt;
        if (payout.Remaining &lt;= 0f)
```

```
            {
                Destroy(payout.Face.gameObject);
                _payouts.RemoveAt(i);
                continue;
            }
            float p = 1f - payout.Remaining / PayoutSeconds;
            payout.Face.rectTransform.anchoredPosition =
                new Vector2(payout.BaseX, payout.BaseY + 70f * p);
            payout.Face.color = new Color(ReadyColor.r, ReadyColor.g, ReadyColor.b, 1f - p);
        }
    }
    // ----- UI construction (programmatic, zero scene authoring) -----
    void BuildUi()
    {
        var canvasGo = new GameObject("OrderCanvas", typeof(Canvas), typeof(CanvasScaler),
            typeof(GraphicRaycaster));
        canvasGo.transform.SetParent(transform, false); // page root owns the canvas
        var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
        canvas.renderMode = RenderMode.ScreenSpaceOverlay;
        canvas.sortingOrder = 1; // above the garden page canvas, strip-local only
        var scaler = canvasGo.GetComponent&lt;CanvasScaler&gt;();
        // U184: ScaleWithScreenSize or the reference frame never drives
        // device scale (raw ConstantPixelSize = shrunken UI on phone).
        scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
        scaler.referenceResolution = new Vector2(ReferenceWidth, ReferenceHeight);
        var safeGo = new GameObject("SafeArea", typeof(RectTransform), typeof(Sy.Core.SafeAreaFitter));
        safeGo.transform.SetParent(canvasGo.transform, false);
        var safe = (RectTransform)safeGo.transform;
        safe.anchorMin = Vector2.zero;
        safe.anchorMax = Vector2.one;
        safe.sizeDelta = Vector2.zero; // full-stretch, fitter squeezes insets
        _safeRoot = safe;
        // T011-b: the wooden order board panel sits behind the strip faces
        // (AR-03, null-safe: art missing keeps the procedural strip look).
        Sy.BloomHaven.UI.PageArt.BuildNode(safe, BoardArtName, 0f, BoardArtY,
            BoardArtWidth, BoardArtHeight, Sy.Core.ArtRuntime.Get(BoardArtKey));
        var title = MakeText(safe, "BoardTitle", 30f, new Vector2(0f, -412f), 680, 36);
        AnchorTop(title.rectTransform);
        title.text = LocalizeService.Get("order.title");
        var hint = MakeText(safe, "BoardHint", 18f, new Vector2(0f, -444f), 680, 28);
        AnchorTop(hint.rectTransform);
        hint.color = new Color(1f, 1f, 1f, 0.75f);
        hint.text = LocalizeService.Get("order.hint");
        for (int i = 0; i &lt; OrderSystem.BoardSize; i++) BuildCard(safe, i);
    }
    void BuildCard(Transform parent, int i)
    {
        float x = (i - 1) * (CardWidth + 14f); // -228 / 0 / +228
        var face = new CardFace();
        var cardGo = new GameObject("OrderCard" + i, typeof(RectTransform), typeof(Image));
```

```
        cardGo.transform.SetParent(parent, false);
        var card = (RectTransform)cardGo.transform;
        card.sizeDelta = new Vector2(CardWidth, CardHeight);
        card.anchoredPosition = new Vector2(x, CardRowY);
        var bg = cardGo.GetComponent&lt;Image&gt;();
        bg.color = CardColor;
        bg.raycastTarget = false; // strip-local: never steal garden taps outside cards
        face.Bg = bg;
        face.Name = MakeText(cardGo.transform, "OrderName", 19f, new Vector2(0f, 70f), 204, 28);
        face.Target = MakeText(cardGo.transform, "OrderTarget", 16f, new Vector2(0f, 34f), 204, 44);
        face.Reward = MakeText(cardGo.transform, "OrderReward", 18f, new Vector2(0f, -8f), 204, 26);
        face.Reward.color = ReadyColor;
        face.Status = MakeText(cardGo.transform, "OrderStatus", 14f, new Vector2(0f, -34f), 204, 22);
        face.Accept = MakeButton(cardGo.transform, "AcceptBtn", "Accept", -74f, -68f, 64f, 34f);
        face.Accept.image.color = new Color(0.36f, 0.52f, 0.42f, 1f);
        face.Complete = MakeButton(cardGo.transform, "CompleteBtn", "Complete", 0f, -68f, 74f, 34f);
        face.Complete.image.color = DimColor;
        face.Drop = MakeButton(cardGo.transform, "DropBtn", "Drop", 72f, -68f, 56f, 34f);
        face.Drop.image.color = new Color(0.42f, 0.36f, 0.36f, 1f);
        CardFace captured = face;
        face.Accept.onClick.AddListener(() =&gt; OnAcceptTapped(captured));
        face.Drop.onClick.AddListener(() =&gt; OnDropTapped(captured));
        face.Complete.onClick.AddListener(() =&gt; OnCompleteTapped(captured));
        _cards.Add(face);
        _lastSatisfied.Add(false);
    }
    static Button MakeButton(Transform parent, string name, string label,
        float x, float y, float w, float h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = new Vector2(x, y);
        var button = go.GetComponent&lt;Button&gt;();
        button.targetGraphic = go.GetComponent&lt;Image&gt;(); // U184: real target graphic
        button.transition = Selectable.Transition.ColorTint;
        var text = MakeText(go.transform, "Label", 14f, Vector2.zero, (int)w - 6, (int)h - 8);
        text.text = label;
        return button;
    }
    static void AnchorTop(RectTransform rt)
    {
        rt.anchorMin = new Vector2(0.5f, 1f);
        rt.anchorMax = new Vector2(0.5f, 1f);
    }
    static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
```

```
            var rt = (RectTransform)go.transform;
            rt.sizeDelta = new Vector2(w, h);
            rt.anchoredPosition = pos;
            var t = go.GetComponent&lt;Text&gt;();
            t.font = HudFont.Resolve();
            t.fontSize = Mathf.RoundToInt(size);
            t.alignment = TextAnchor.MiddleCenter;
            t.color = Color.white;
            t.raycastTarget = false; // never steal a button tap
            return t;
        }
    }
}
// OrderModel.cs
using System;
using System.Collections.Generic;
namespace Sy.BloomHaven.Orders
{
    /// &lt;summary&gt;One order instance living on the board (save.orders face).&lt;/summary&gt;
    [Serializable]
    public sealed class OrderState
    {
        public string DefId = string.Empty;
        public bool Accepted;
        /// &lt;summary&gt;Live-play clock seconds when the order was accepted (X1080 risk
        /// window); -1 = never accepted, no window. The sitting slice enforces it:
        /// the live clock strictly past stamp + RiskWindowSec = expired (swept off
        /// the board, CompleteOrder rejects it — the window IS the deadline).&lt;/summary&gt;
        public double AcceptedAtSec = -1.0;
    }
    /// &lt;summary&gt;
    /// Whole order-board persistent state (save.orders face). The board still only
    /// mutates through explicit calls inside live play (offline never refreshes —
    /// requirement doc S2 row); accepted orders carry an X1080 risk window whose
    /// close is enforced (window close unsatisfied = lost sitting — the broken
    /// cozy promise; the sweep books one LostCount per lost sitting).
    /// &lt;/summary&gt;
    [Serializable]
    public sealed class OrdersState
    {
        public List&lt;OrderState&gt; Board = new List&lt;OrderState&gt;();
        public int CompletedCount;
        /// &lt;summary&gt;Accepted orders whose risk window closed unsatisfied (X1080 loss
        /// face, F1/F6 fix): one per swept/abandoned-expired sitting.&lt;/summary&gt;
        public int LostCount;
    }
    /// &lt;summary&gt;
    /// One bouquet-cut beat payload (S2 order-delivery staging depth): the slot
    /// index the satisfying tap consumed and the species that stood there - the
    /// delivery view stages its goodbye fade on exactly these (the cut itself
```

```
    /// stays inside CompleteOrder; this only exposes what was cut).
    /// &lt;/summary&gt;
    public readonly struct BouquetCut
    {
        public readonly int SlotIndex;
        public readonly string SpeciesId;
        public BouquetCut(int slotIndex, string speciesId)
        {
            SlotIndex = slotIndex;
            SpeciesId = speciesId;
        }
    }
    /// &lt;summary&gt;Completion outcome; Gained is the amount actually credited (wallet cap clips it).
    /// Cut is the bouquet-delivery payload (S2 staging depth): the slots consumed by the
    /// satisfying tap, lowest-index-first, empty on every failure face.&lt;/summary&gt;
    public readonly struct OrderResult
    {
        public readonly bool Ok;
        public readonly int Gained;
        public readonly int WalletAfter;
        public readonly int FlippedCount;
        public readonly BouquetCut[] Cut;
        public OrderResult(bool ok, int gained, int walletAfter, int flippedCount)
            : this(ok, gained, walletAfter, flippedCount, null) { }
        public OrderResult(bool ok, int gained, int walletAfter, int flippedCount, BouquetCut[] cut)
        {
            Ok = ok;
            Gained = gained;
            WalletAfter = walletAfter;
            FlippedCount = flippedCount;
            Cut = cut ?? EmptyCut;
        }
        public static readonly BouquetCut[] EmptyCut = new BouquetCut[0];
    }
}
// OrderSystem.cs
using System;
using System.Collections.Generic;
using Sy.BloomHaven.Garden;
namespace Sy.BloomHaven.Orders
{
    /// &lt;summary&gt;
    /// Renovation order rules engine (M4 + T012-g c1 bouquet semantics). Pure C#
    /// over the garden state: the board offers pool orders, targets are judged on
    /// SIMULTANEOUS IN-BLOOM slots (a Growing slot never counts; a Bloom slot past
    /// its keep window counts nowhere - the plant-then-live double channel is dead,
    /// X1291 freeze M2), completing a satisfied order pays petals, cuts the
    /// satisfying blooms (bouquet delivery, cut-for-consumption) and flips its
    /// block ground, abandoning costs nothing (R112 cozy law). No time-based
    /// refresh of the board itself — it only changes through explicit calls, so
```

```
/// offline play leaves it untouched; the board entries run the wilt sweep so
/// the board a player opens never holds stale stock. Accepted orders carry
/// an X1080 risk-window stamp, and the sitting slice (F1/F6 fix slice 2)
/// enforces it: an accepted order strictly past its window close is expired
/// (the window IS the deadline — the close tick itself stays in-window as
/// the edge save). Expired orders are swept off the board inside the
/// explicit order calls (one LostCount per, lost ids returned as feedback);
/// CompleteOrder rejects them even when satisfied; unaccepted offers never
/// expire (no commitment, no window — R112 cozy law).
/// &lt;/summary&gt;
public sealed class OrderSystem
{
    public const int BoardSize = 3;  // concurrent offers on the order board
    // ----- X1080 risk window (FQ slice 3a freeze mirror; the gate sitting judge
    // freezes the same numbers — keep both sides in lockstep or the measured and
    // enforced windows diverge) -----
    public const double RiskK = 0.3;         // window = MinSlots * GrowSeconds * K + slack
    public const double RiskSlackSec = 8.0;
    public GardenSystem Garden { get; }
    public OrdersState State { get; }
    readonly Random _rng;
    public OrderSystem(GardenSystem garden, OrdersState state = null, Random rng = null)
    {
        Garden = garden ?? throw new ArgumentNullException("garden");
        State = state ?? new OrdersState();
        _rng = rng ?? new Random();
    }
    /// &lt;summary&gt;Top the board back up to BoardSize with pool orders not already
    /// shown (page-open driven, idempotent when full). The expired sweep and the
    /// wilt sweep run first so the board a player interacts with never holds a
    /// lost order or judges against stale stock (T012-g c1 board-entry mirror
    /// of the X1290 SweepExpired pattern).&lt;/summary&gt;
    public void EnsureBoard()
    {
        SweepExpired();
        Garden.SweepWilt();
        while (State.Board.Count &lt; BoardSize)
        {
            OrderDef def = DrawNewDef();
            if (def == null) break;  // pool exhausted before the board filled (cannot happen: 12 &gt; 3)
            State.Board.Add(new OrderState { DefId = def.Id });
        }
    }
    /// &lt;summary&gt;
    /// Full re-roll of the board (AD_ORDER_REFRESH mechanism face; the rewarded-ad gate
    /// itself is S2 wiring — the data roll is free). The expired sweep and the wilt
    /// sweep run first so window losses book as losses and stale blooms free their
    /// slots; the player-chosen wipe of the rest stays free.
    /// &lt;/summary&gt;
    public void RefreshBoard()
```

```
{
    SweepExpired();
    Garden.SweepWilt();
    State.Board.Clear();
    EnsureBoard();
}
/// &lt;summary&gt;Accept an offered order (commitment face for the board UI; any number
/// may be in flight). The first acceptance stamps the live-play clock for the X1080
/// risk window — one window per sitting: a re-accept never re-stamps (the deadline
/// cannot be refreshed), and an expired order can no longer be committed.&lt;/summary&gt;
public bool AcceptOrder(string defId)
{
    OrderState order = FindOnBoard(defId);
    if (order == null) return false;
    if (order.Accepted) return true;  // idempotent re-commit, window keeps its first-accept stamp
    order.Accepted = true;
    order.AcceptedAtSec = Garden.Now;
    return true;
}
/// &lt;summary&gt;
/// X1080 enforcement face (sitting slice): true when the live-play clock is
/// STRICTLY past the order's risk-window close (the close tick itself is
/// still in-window — the edge save). Pure time face: expiry never looks at
/// satisfaction, the window IS the deadline. Unaccepted orders and unknown
/// ids are never expired (no commitment, no window).
/// &lt;/summary&gt;
public bool IsExpired(string defId)
{
    return IsExpired(FindOnBoard(defId));
}
bool IsExpired(OrderState order)
{
    if (order == null || !order.Accepted || order.AcceptedAtSec &lt; 0.0) return false;
    OrderDef def = OrderTable.Get(order.DefId);
    return def != null &amp;&amp; Garden.Now &gt; order.AcceptedAtSec + RiskWindowSec(def);
}
/// &lt;summary&gt;
/// Enforcement sweep (sitting slice): removes every expired accepted order
/// from the board, books one LostCount per removal and returns the lost
/// defIds — the lost feedback face consumers (board UI, sitting judge)
/// render. Keeps the M4 law: only explicit order calls ever mutate the
/// board, no timer drives this.
/// &lt;/summary&gt;
public List&lt;string&gt; SweepExpired()
{
    var lost = new List&lt;string&gt;();
    for (int i = State.Board.Count - 1; i &gt;= 0; i--)
    {
        if (!IsExpired(State.Board[i])) continue;
        lost.Add(State.Board[i].DefId);
```

```
            State.Board.RemoveAt(i);
            State.LostCount++;
        }
        return lost;
    }
    /// &lt;summary&gt;
    /// Risk window seconds for an order def (X1080 freeze mirror of the gate sitting
    /// formula): MinSlots * grow-seconds of the target's resolved species * K + slack.
    /// Species keys resolve by id; family keys resolve to their first-table species
    /// (gate SpeciesOfFamily parity); DualColor takes the slower of its two families.
    /// Unknown keys fall back to 3.0s grow (gate GrowSecOf parity).
    /// &lt;/summary&gt;
    public static double RiskWindowSec(OrderDef def)
    {
        if (def == null) return RiskSlackSec;
        OrderTarget target = def.Target;
        double grow = GrowSecOfKey(target.Primary);
        if (target.Kind == OrderTargetKind.DualColor)
            grow = Math.Max(grow, GrowSecOfKey(target.Secondary));
        return target.MinSlots * grow * RiskK + RiskSlackSec;
    }
    static double GrowSecOfKey(string key)
    {
        for (int i = 0; i &lt; SpeciesTable.Count; i++)
        {
            SpeciesDef sd = SpeciesTable.Get(i);
            if (sd != null &amp;&amp;
                (string.Equals(sd.Id, key, StringComparison.Ordinal) ||
                 string.Equals(sd.ColorFamily, key, StringComparison.Ordinal)))
                return sd.GrowSeconds;
        }
        return 3.0;
    }
    /// &lt;summary&gt;Live judgment for a pool order id; unknown ids are never satisfied.&lt;/summary&gt;
    public bool IsSatisfied(string defId)
    {
        OrderDef def = OrderTable.Get(defId);
        return def != null &amp;&amp; Judge(def.Target);
    }
    /// &lt;summary&gt;
    /// Complete a satisfied board order: pay petals (clip at wallet cap), flip its block
    /// ground to renovated, cut the satisfying in-bloom flowers (T012-g c1 bouquet
    /// delivery - the blooms the tap judged are consumed, no harvest payout on top of
    /// the order reward), drop it from the board. Unknown, off-board, expired or
    /// unsatisfied orders change nothing — expiry is the time face (the window IS the
    /// deadline, satisfaction does not revive a lost sitting). Acceptance is not
    /// required — judgment is state truth.
    /// &lt;/summary&gt;
    public OrderResult CompleteOrder(string defId)
    {
```

```
    OrderDef def = OrderTable.Get(defId);
    if (def == null || !IsOnBoard(defId) || IsExpired(defId) || !Judge(def.Target))
        return new OrderResult(false, 0, Garden.State.Petals, 0);
    int credited = Math.Min(GardenSystem.WalletCap - Garden.State.Petals, def.RewardPetals);
    if (credited &lt; 0) credited = 0;
    Garden.State.Petals += credited;
    int flipped = 0;
    int[] blockSlots = OrderTable.GetBlockSlots(def.FlipBlock);
    for (int i = 0; i &lt; blockSlots.Length; i++)
    {
        SlotState slot = Garden.State.Slots[blockSlots[i]];
        if (!slot.Renovated) { slot.Renovated = true; flipped++; }
    }
    BouquetCut[] cut = CutBlooms(def.Target);
    RemoveFromBoard(defId);
    State.CompletedCount++;
    return new OrderResult(true, credited, Garden.State.Petals, flipped, cut);
}
/// &lt;summary&gt;
/// R112 cozy law: abandoning an order costs nothing and penalizes nothing — no petal
/// fee, no garden damage, no lockout. The board slot stays empty until topped up.
/// An abandoned order that had already expired still books its LostCount (the loss
/// happened at the window close; the tap only clears the corpse).
/// &lt;/summary&gt;
public bool AbandonOrder(string defId)
{
    OrderState order = FindOnBoard(defId);
    if (order == null) return false;
    if (IsExpired(order)) State.LostCount++;
    return RemoveFromBoard(defId);
}
public bool IsOnBoard(string defId)
{
    return FindOnBoard(defId) != null;
}
OrderState FindOnBoard(string defId)
{
    for (int i = 0; i &lt; State.Board.Count; i++)
        if (string.Equals(State.Board[i].DefId, defId, StringComparison.Ordinal))
            return State.Board[i];
    return null;
}
bool RemoveFromBoard(string defId)
{
    for (int i = 0; i &lt; State.Board.Count; i++)
    {
        if (!string.Equals(State.Board[i].DefId, defId, StringComparison.Ordinal)) continue;
        State.Board.RemoveAt(i);
        return true;
    }
```

```
    return false;
}
OrderDef DrawNewDef()
{
    var free = new List&lt;OrderDef&gt;();
    for (int i = 0; i &lt; OrderTable.PoolCount; i++)
    {
        OrderDef def = OrderTable.Get(i);
        if (def == null || IsOnBoard(def.Id)) continue;
        free.Add(def);
    }
    return free.Count == 0 ? null : free[_rng.Next(free.Count)];
}
/// &lt;summary&gt;
/// Judge a target against the garden (T012-g c1 simultaneous face): only
/// blooms still inside their keep window count — N target flowers must
/// stand in-bloom CONCURRENTLY at the tap instant; dual-color needs at
/// least one of each family and the combined minimum.
/// &lt;/summary&gt;
bool Judge(OrderTarget target)
{
    int primary = 0, secondary = 0;
    SlotState[] slots = Garden.State.Slots;
    for (int i = 0; i &lt; slots.Length; i++)
    {
        if (!InBloom(slots[i])) continue;
        SpeciesDef species = SpeciesTable.Get(slots[i].SpeciesId);
        if (species == null) continue;
        if (Matches(target, species, true)) primary++;
        if (target.Kind == OrderTargetKind.DualColor &amp;&amp; Matches(target, species, false)) secondary++;
    }
    switch (target.Kind)
    {
        case OrderTargetKind.ColorFamily:
        case OrderTargetKind.Species:
            return primary &gt;= target.MinSlots;
        case OrderTargetKind.DualColor:
            return primary &gt;= 1 &amp;&amp; secondary &gt;= 1 &amp;&amp; primary + secondary &gt;= target.MinS
        default:
            return false;
    }
}
/// &lt;summary&gt;
/// Simultaneous in-bloom face (T012-g c1): a Growing slot never counts;
/// a Bloom slot counts only while its keep window (BloomedAt +
/// BloomKeepSec, the same math Tick enforces) still holds — the judge
/// reads the time face directly so an unopened board can never
/// stockpile sittings on stale blooms.
/// &lt;/summary&gt;
bool InBloom(SlotState slot)
```

```
{
    if (slot == null || slot.Stage != SlotStage.Bloom) return false;
    SpeciesDef species = SpeciesTable.Get(slot.SpeciesId);
    if (species == null) return false;
    return Garden.Now - slot.BloomedAt &lt; GardenSystem.BloomKeepSec(species);
}
/// &lt;summary&gt;
/// Bouquet-cut (T012-g c1 M2 + S2 staging depth): the satisfying tap consumes
/// the in-bloom flowers it judged — exactly the target's needed count returns
/// to empty (cut-for-bouquet, no harvest payout on top of the order
/// reward). Deterministic lowest-slot-index-first; dual cuts one bloom
/// of each family first, then fills the combined minimum from the rest.
/// Runs only after Judge passed on the same call, so the needed
/// matches always exist. Returns the cut payload (slot index + the
/// species that stood there) in cut order — the delivery view stages
/// its goodbye fade on exactly these.
/// &lt;/summary&gt;
BouquetCut[] CutBlooms(OrderTarget target)
{
    var cut = new List&lt;BouquetCut&gt;();
    SlotState[] slots = Garden.State.Slots;
    int remaining = target.MinSlots;
    if (target.Kind == OrderTargetKind.DualColor)
    {
        bool primaryCut = false, secondaryCut = false;
        for (int i = 0; i &lt; slots.Length &amp;&amp; remaining &gt; 0; i++)
        {
            if (!InBloom(slots[i])) continue;
            SpeciesDef species = SpeciesTable.Get(slots[i].SpeciesId);
            if (species == null) continue;
            if (!primaryCut &amp;&amp; Matches(target, species, true))
            { RecordCut(cut, slots[i], i, species); primaryCut = true; remaining--; continue; }
            if (!secondaryCut &amp;&amp; Matches(target, species, false))
            { RecordCut(cut, slots[i], i, species); secondaryCut = true; remaining--; continue; }
        }
    }
    for (int i = 0; i &lt; slots.Length &amp;&amp; remaining &gt; 0; i++)
    {
        if (!InBloom(slots[i])) continue;
        SpeciesDef species = SpeciesTable.Get(slots[i].SpeciesId);
        if (species == null) continue;
        bool match = Matches(target, species, true);
        if (!match &amp;&amp; target.Kind == OrderTargetKind.DualColor)
            match = Matches(target, species, false);
        if (!match) continue;
        RecordCut(cut, slots[i], i, species); remaining--;
    }
    return cut.ToArray();
}
static void RecordCut(List&lt;BouquetCut&gt; cut, SlotState slot, int index, SpeciesDef species)
```

```
        {
            cut.Add(new BouquetCut(index, slot.SpeciesId));
            CutSlot(slot);
        }
        static void CutSlot(SlotState slot)
        {
            slot.SpeciesId = string.Empty;
            slot.PlantedAt = 0;
            slot.BloomedAt = 0;
            slot.Stage = SlotStage.Empty;
        }
        static bool Matches(OrderTarget target, SpeciesDef species, bool primaryFace)
        {
            switch (target.Kind)
            {
                case OrderTargetKind.ColorFamily:
                    return string.Equals(species.ColorFamily, target.Primary, StringComparison.Ordinal);
                case OrderTargetKind.Species:
                    return string.Equals(species.Id, target.Primary, StringComparison.Ordinal);
                case OrderTargetKind.DualColor:
                    string want = primaryFace ? target.Primary : target.Secondary;
                    return string.Equals(species.ColorFamily, want, StringComparison.Ordinal);
                default:
                    return false;
            }
        }
    }
}
// OrderTable.cs
using System;
namespace Sy.BloomHaven.Orders
{
    /// &lt;summary&gt;Target shape of a renovation order (requirement doc order schema: target_spec).&lt;/summary&gt;
    public enum OrderTargetKind
    {
        ColorFamily,   // at least MinSlots alive flowers of one color family
        Species,       // at least MinSlots alive flowers of one species
        DualColor      // both color families present, combined at least MinSlots
    }
    /// &lt;summary&gt;Order target_spec face (frozen in OrderTable, judged by OrderSystem).&lt;/summary&gt;
    public sealed class OrderTarget
    {
        public readonly OrderTargetKind Kind;
        public readonly string Primary;   // color family id, or species id for Species targets
        public readonly string Secondary;  // second color family id (DualColor only)
        public readonly int MinSlots;
        public OrderTarget(OrderTargetKind kind, string primary, string secondary, int minSlots)
        {
            Kind = kind;
            Primary = primary;
```

```
            Secondary = secondary;
            MinSlots = minSlots;
        }
    }
    /// &lt;summary&gt;
    /// Renovation order definition, frozen pool of 12 (requirement doc L79 anchor).
    /// Rewards sit in the 3-8 petals band; difficulty slopes single-target -&gt; dual-target.
    /// &lt;/summary&gt;
    public sealed class OrderDef
    {
        public readonly string Id;
        public readonly string NameEn;
        public readonly string NameKey;
        public readonly OrderTarget Target;
        public readonly int RewardPetals;
        public readonly int FlipBlock;
        public OrderDef(string id, string nameEn, string nameKey, OrderTarget target,
            int rewardPetals, int flipBlock)
        {
            Id = id;
            NameEn = nameEn;
            NameKey = nameKey;
            Target = target;
            RewardPetals = rewardPetals;
            FlipBlock = flipBlock;
        }
    }
    /// &lt;summary&gt;
    /// Frozen order pool (M4 renovation orders). The 16 preset slots form four renovation
    /// blocks of four slots (2x2 quadrants of the 4x4 grid); completing an order flips the
    /// ground of its block from wasteland to flower bed. No deadline by design (cozy law).
    /// &lt;/summary&gt;
    public static class OrderTable
    {
        public const string Ord01 = "ORD-01";
        public const string Ord12 = "ORD-12";
        public const int PoolCount = 12;
        public const int BlockCount = 4;      // 16 slots = 4 renovation blocks
        public const int SlotsPerBlock = 4;
        static readonly OrderDef[] Pool =
        {
            new OrderDef(Ord01, "Bluebell Bed", "order_ORD01",
                new OrderTarget(OrderTargetKind.ColorFamily, "blue", null, 3), 3, 0),
            new OrderDef("ORD-02", "Sunshelf", "order_ORD02",
                new OrderTarget(OrderTargetKind.Species, "SP02", null, 2), 3, 0),
            new OrderDef("ORD-03", "Blue &amp;amp; Gold Blend", "order_ORD03",
                new OrderTarget(OrderTargetKind.DualColor, "blue", "yellow", 5), 5, 0),
            new OrderDef("ORD-04", "Sunny Plot", "order_ORD04",
                new OrderTarget(OrderTargetKind.ColorFamily, "yellow", null, 3), 4, 1),
            new OrderDef("ORD-05", "Heartrow", "order_ORD05",
```

```
                new OrderTarget(OrderTargetKind.ColorFamily, "red", null, 2), 4, 1),
            new OrderDef("ORD-06", "Dusk Garden", "order_ORD06",
                new OrderTarget(OrderTargetKind.DualColor, "blue", "purple", 5), 5, 1),
            new OrderDef("ORD-07", "Mist Bed", "order_ORD07",
                new OrderTarget(OrderTargetKind.ColorFamily, "purple", null, 3), 4, 2),
            new OrderDef("ORD-08", "Rose Arbor", "order_ORD08",
                new OrderTarget(OrderTargetKind.Species, "SP03", null, 2), 5, 2),
            new OrderDef("ORD-09", "Ember Duet", "order_ORD09",
                new OrderTarget(OrderTargetKind.DualColor, "red", "purple", 5), 6, 2),
            new OrderDef("ORD-10", "Sunset Border", "order_ORD10",
                new OrderTarget(OrderTargetKind.DualColor, "yellow", "red", 6), 6, 3),
            new OrderDef("ORD-11", "Lavender Lane", "order_ORD11",
                new OrderTarget(OrderTargetKind.Species, "SP04", null, 3), 6, 3),
            new OrderDef(Ord12, "Starfield Mosaic", "order_ORD12",
                new OrderTarget(OrderTargetKind.DualColor, "blue", "red", 8), 8, 3),
        };
        /// &lt;summary&gt;Index lookup (0..PoolCount-1); out of range returns null.&lt;/summary&gt;
        public static OrderDef Get(int index) =&gt;
            (index &gt;= 0 &amp;&amp; index &lt; Pool.Length) ? Pool[index] : null;
        /// &lt;summary&gt;Id lookup; unknown id returns null (callers treat null as invalid).&lt;/summary&gt;
        public static OrderDef Get(string id)
        {
            for (int i = 0; i &lt; Pool.Length; i++)
                if (string.Equals(Pool[i].Id, id, StringComparison.Ordinal)) return Pool[i];
            return null;
        }
        /// &lt;summary&gt;
        /// Preset slots composing a renovation block (4x4 row-major grid, 2x2 quadrants).
        /// Out-of-range block returns null.
        /// &lt;/summary&gt;
        public static int[] GetBlockSlots(int blockIndex)
        {
            switch (blockIndex)
            {
                case 0: return new[] { 0, 1, 4, 5 };
                case 1: return new[] { 2, 3, 6, 7 };
                case 2: return new[] { 8, 9, 12, 13 };
                case 3: return new[] { 10, 11, 14, 15 };
                default: return null;
            }
        }
    }
}
// SaveHost.cs
using System;
using UnityEngine;
namespace Sy.BloomHaven.Save
{
    /// &lt;summary&gt;
    /// App-lifetime save host (T007 slice b): loads + settles the save exactly once
```

```
    /// per process before any page reads state (boot wires it, pages consume
    /// Current). LastSettlement feeds the S01 welcome-back bubble. Pages drive
    /// persistence through SaveNow on a periodic interval plus pause/quit hooks.
    /// &lt;/summary&gt;
    public static class SaveHost
    {
        static SaveSystem _current;
        public static OfflineSettlement? LastSettlement { get; private set; }
        public static SaveSystem Current =&gt; Ensure();
        /// &lt;summary&gt;
        /// Load-or-fresh, bridge the clock domain, settle the away window once.
        /// Fresh saves settle to zero (no free petals on first launch).
        /// &lt;/summary&gt;
        public static SaveSystem Ensure()
        {
            if (_current != null) return _current;
            double uptimeNow = Time.realtimeSinceStartupAsDouble;
            SaveFile file = SaveStore.Load();
            if (file == null)
            {
                _current = new SaveSystem(); // fresh face: absent or unreadable save
                LastSettlement = _current.Settle(uptimeNow); // stamps the anchor, accrues nothing
                return _current;
            }
            double wallAway = Math.Max(0, SaveStore.WallSeconds() - file.WallSavedAt); // backward wall clock = 0 away,
            _current = new SaveSystem(file.State);
            LastSettlement = SaveStore.Resume(_current, uptimeNow, file.UptimeSavedAt, wallAway);
            return _current;
        }
        /// &lt;summary&gt;Periodic / pause / quit persistence (live path).&lt;/summary&gt;
        public static void SaveNow(double uptimeNow)
        {
            if (_current == null) return;
            SaveStore.Persist(_current, uptimeNow);
        }
    }
}
// SaveModel.cs
using System;
using System.Collections.Generic;
namespace Sy.BloomHaven.Save
{
    /// &lt;summary&gt;Codex collection entry (save.codex face, S5). Unlocked once at first
    /// bloom; BloomCount keeps rising for the codex card (T010-a face).&lt;/summary&gt;
    [Serializable]
    public sealed class CodexEntry
    {
        public string SpeciesId = string.Empty;
        public double UnlockedAt;
        public int BloomCount;
```

```
    }
    /// &lt;summary&gt;One rewarded-ad capsule record (save.adcaps face, S6). U149: 12/day hard cap enforced by the futu
    [Serializable]
    public sealed class AdCapsule
    {
        public string Type = string.Empty;
        public double OfferedAt;
    }
    /// &lt;summary&gt;
    /// Whole-profile save state (M6): five faces = slots + wallet (hosted by GardenState,
    /// see its class doc) / orders / codex / adcaps, plus LastSeenAt as the monotonic-clock
    /// anchor for offline accrual (M6: away seconds -&gt; petals, 60s -&gt; 6, cap 10, E4 h5).
    /// &lt;/summary&gt;
    [Serializable]
    public sealed class SaveState
    {
        public double LastSeenAt;
        public Garden.GardenState Garden = new Garden.GardenState();   // save.slots + save.wallet faces
        public Orders.OrdersState Orders = new Orders.OrdersState();    // save.orders face
        public List&lt;CodexEntry&gt; Codex = new List&lt;CodexEntry&gt;();        // save.codex face
        public List&lt;AdCapsule&gt; AdCaps = new List&lt;AdCapsule&gt;();          // save.adcaps face
    }
    /// &lt;summary&gt;Offline settlement outcome (M6). Raw is the uncapped accrual, Accrued is what was actually credit
    public readonly struct OfflineSettlement
    {
        public readonly double AwaySeconds;
        public readonly int Raw;
        public readonly int Accrued;
        public readonly bool Capped;
        public readonly int WalletAfter;
        public OfflineSettlement(double awaySeconds, int raw, int accrued, bool capped, int walletAfter)
        {
            AwaySeconds = awaySeconds;
            Raw = raw;
            Accrued = accrued;
            Capped = capped;
            WalletAfter = walletAfter;
        }
    }
}
// SaveStore.cs
using System;
using UnityEngine;
namespace Sy.BloomHaven.Save
{
    /// &lt;summary&gt;
    /// On-disk save wrapper (T007 slice b). The five faces ride SaveState; the two
    /// anchors translate the uptime clock domain across process restarts:
    /// WallSavedAt is wall-clock Unix seconds at save time, UptimeSavedAt is
    /// Time.realtimeSinceStartupAsDouble at save time. Version gates decode:
```

```
/// anything unexpected heals to a fresh save (malformed-input cure law).
/// &lt;/summary&gt;
[Serializable]
public sealed class SaveFile
{
    public int Version = 1;
    public double WallSavedAt;
    public double UptimeSavedAt;
    public SaveState State;
}
/// &lt;summary&gt;
/// PlayerPrefs persistence + cross-launch clock bridge (T007 slice b). The away
/// window is measured on the wall clock (the only clock that survives a
/// restart); growth replays through the SAME Settle path as live play by
/// remapping PlantedAt so the maturity a slot had at save time is preserved,
/// then letting Settle(now) advance it by exactly the away seconds. The engine
/// clock stays in the live uptime domain, so plants made after a revisit grow
/// on live ticks with zero future-dated drift.
/// &lt;/summary&gt;
public static class SaveStore
{
    public const string PlayerPrefsKey = "g19_save_v1";
    public const int SaveVersion = 1;
    /// &lt;summary&gt;Wall-clock Unix seconds (restart-surviving clock face).&lt;/summary&gt;
    public static double WallSeconds()
        =&gt; DateTimeOffset.UtcNow.ToUnixTimeSeconds();
    /// &lt;summary&gt;Encode the live save into the on-disk wrapper (pure, injected clocks).&lt;/summary&gt;
    public static string Encode(SaveSystem save, double uptimeNow, double wallNowSeconds)
    {
        save.MarkSeen(uptimeNow); // anchor stamps at the save instant
        var file = new SaveFile
        {
            Version = SaveVersion,
            WallSavedAt = wallNowSeconds,
            UptimeSavedAt = uptimeNow,
            State = save.State
        };
        return JsonUtility.ToJson(file);
    }
    /// &lt;summary&gt;Decode a save payload; null heals any malformed or unknown-version input.&lt;/summary&gt;
    public static SaveFile Decode(string json)
    {
        if (string.IsNullOrEmpty(json)) return null;
        try
        {
            var file = JsonUtility.FromJson&lt;SaveFile&gt;(json);
            if (file == null || file.State == null || file.Version != SaveVersion) return null;
            if (file.WallSavedAt &lt;= 0 || file.UptimeSavedAt &lt; 0) return null;
            if (file.State.Garden == null || file.State.Garden.Slots == null
                || file.State.Garden.Slots.Length != Garden.GardenSystem.SlotCount) return null;
```

```
                if (file.State.Orders == null || file.State.Codex == null || file.State.AdCaps == null) return null;
                return file;
            }
            catch
            {
                return null; // malformed-input cure law: unreadable save = fresh start
            }
        }
        /// &lt;summary&gt;
        /// Bridge a decoded save into the live session and settle the away window
        /// through the SAME Settle path as live play (single-path law). Slots remap
        /// so their maturity at save time is preserved, then Settle advances growth
        /// by exactly wallAway seconds. The anchor is remapped to now-wallAway so
        /// Settle's internal away equals the wall gap; a fresh face (LastSeenAt=0)
        /// keeps the zero guard - no free petals on first launch. T012-g c1 cozy
        /// law: in-bloom slots re-stamp their keep-window clock by the extra
        /// wallAway so the away gap never advances wilt (offline rewards promise
        /// untouched); Growing slots keep advancing by wallAway as before.
        /// &lt;/summary&gt;
        public static OfflineSettlement Resume(SaveSystem save, double uptimeNow,
            double savedUptimeAt, double wallAway)
        {
            double shift = uptimeNow - savedUptimeAt - wallAway;
            var slots = save.State.Garden.Slots;
            for (int i = 0; i &lt; slots.Length; i++)
            {
                slots[i].PlantedAt += shift;
                if (slots[i].Stage == Garden.SlotStage.Bloom)
                    slots[i].BloomedAt += shift + wallAway; // keep age frozen across the away gap
            }
            if (save.State.LastSeenAt &gt; 0) save.State.LastSeenAt = uptimeNow - wallAway;
            return save.Settle(uptimeNow); // accrual + offline growth + anchor stamp, one path
        }
        /// &lt;summary&gt;Persist to PlayerPrefs (live path).&lt;/summary&gt;
        public static void Persist(SaveSystem save, double uptimeNow)
        {
            PlayerPrefs.SetString(PlayerPrefsKey, Encode(save, uptimeNow, WallSeconds()));
            PlayerPrefs.Save();
        }
        /// &lt;summary&gt;Load from PlayerPrefs; null when absent or malformed.&lt;/summary&gt;
        public static SaveFile Load()
            =&gt; Decode(PlayerPrefs.GetString(PlayerPrefsKey, string.Empty));
    }
}
// SaveSystem.cs
using System;
namespace Sy.BloomHaven.Save
{
    /// &lt;summary&gt;
    /// Save facade + offline settlement engine (M6/T007). Owns the five-face SaveState and
```

```
    /// wires the rules engines onto it (GardenSystem over save.slots+wallet, OrderSystem over
    /// save.orders; T010-b adds CodexSystem over save.codex + IaaSystem over save.adcaps so the
    /// live app records codex unlocks on every bloom and shares one ad ledger with the boost
    /// buttons). Offline accrual: floor(away/10) petals, capped at 10 (E4 h5: 60s -&gt; 6),
    /// then the wallet cap 999 clips the credit. Growth resume rides the same monotonic
    /// Tick path as live play (T003 law: stage math is always (now - plantedAt), no new path).
    /// Orders never mutate through time (S2 row: offline never refreshes).
    /// &lt;/summary&gt;
    public sealed class SaveSystem
    {
        public const double AccrualSecondsPerPetal = 10.0;  // 60s away -&gt; 6 petals (E4 h5)
        public const int AccrualCap = 10;                   // accrual hard cap (E4 h5)
        public SaveState State { get; }
        public Garden.GardenSystem Garden { get; }
        public Orders.OrderSystem Orders { get; }
        public Codex.CodexSystem Codex { get; }             // T010-b: bloom-subscribed collection engine
        public Iaa.IaaSystem Iaa { get; }                   // T010-b: single ad-capsule ledger face
        public SaveSystem(SaveState state = null, Random rng = null)
        {
            State = state ?? new SaveState();
            Garden = new Garden.GardenSystem(State.Garden);
            Orders = new Orders.OrderSystem(Garden, State.Orders, rng);
            Codex = new Codex.CodexSystem(Garden, State);
            Iaa = new Iaa.IaaSystem(State);
        }
        /// &lt;summary&gt;
        /// Settle a return after being away: accrue petals for the gap since LastSeenAt, resume
        /// offline growth via the shared Tick path, then stamp the new anchor. A fresh save
        /// (LastSeenAt == 0) settles to zero — no free petals on first launch.
        /// &lt;/summary&gt;
        public OfflineSettlement Settle(double nowSeconds)
        {
            double away = State.LastSeenAt &gt; 0 ? Math.Max(0, nowSeconds - State.LastSeenAt) : 0;
            int raw = (int)Math.Floor(away / AccrualSecondsPerPetal);
            bool capped = raw &gt; AccrualCap;
            int accrued = Math.Min(raw, AccrualCap);
            int credited = Math.Min(Sy.BloomHaven.Garden.GardenSystem.WalletCap - State.Garden.Petals, accrued);
            if (credited &lt; 0) credited = 0;
            State.Garden.Petals += credited;
            Garden.Tick(nowSeconds); // offline growth resume, same monotonic path as live play
            if (nowSeconds &gt; State.LastSeenAt) State.LastSeenAt = nowSeconds; // anchor never regresses (backward clo
            return new OfflineSettlement(away, raw, credited, capped, State.Garden.Petals);
        }
        /// &lt;summary&gt;Stamp the anchor without accruing (periodic save / graceful quit).&lt;/summary&gt;
        public void MarkSeen(double nowSeconds)
        {
            if (nowSeconds &gt;= State.LastSeenAt) State.LastSeenAt = nowSeconds;
        }
    }
}
```

```
// ShareCodec.cs
using System;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Orders;
namespace Sy.BloomHaven.Share
{
    /// &lt;summary&gt;
    /// M5 offline garden share code (X032 doctrine: pure local, zero server).
    /// compose: full garden face -&gt; "BH1|&lt;16 slot chars&gt;|&lt;4 block chars&gt;|&lt;petals&gt;".
    /// Slot char: '0'=empty, '1'..'4'=SP01..SP04 growing, 'A'..'D'=SP01..SP04 bloom.
    /// Block char: '0'=wasteland, '1'=renovated (uniform per block, C4 idempotence law).
    /// Petals: decimal 0..999 (wallet cap), the bragging number on the visitor side.
    /// PlantedAt is deliberately NOT encoded (time-relative); growing slots decode with
    /// PlantedAt=0 so they bloom on the visitor's own clock (living-garden entrance).
    /// SelectedSpeciesId is player-local UI state, not garden content - not encoded.
    /// DecodeShare returns null on any malformed input (cozy law: never throw at a visitor).
    /// &lt;/summary&gt;
    public static class ShareCodec
    {
        public const string Prefix = "BH1";
        public const int MaxShareChars = 96;   // north star #4 cap (requirement doc, assert p2)
        const int Sections = 4;                // prefix | slots | blocks | petals
        /// &lt;summary&gt;Information metric for the share_compose telemetry event:
        /// log2 of the distinct encodable state space (16 slots x 9 states + 4 block bits + petals 0..999).&lt;/summary
        public static double StateSpaceBits =&gt;
            GardenSystem.SlotCount * Math.Log(9.0, 2.0) + OrderTable.BlockCount + Math.Log(1000.0, 2.0);
        /// &lt;summary&gt;Compose the shareable garden face. Never throws; unknown species encode as empty.&lt;/summary
        public static string ComposeShare(GardenState state)
        {
            if (state == null) return null;
            var slots = new char[GardenSystem.SlotCount];
            for (int i = 0; i &lt; slots.Length; i++)
            {
                SlotState slot = state.Slots[i];
                int speciesIndex = SpeciesIndexOf(slot == null ? string.Empty : slot.SpeciesId);
                if (speciesIndex &lt; 0) { slots[i] = '0'; continue; }
                if (slot.Stage == SlotStage.Bloom) slots[i] = (char)('A' + speciesIndex);
                else if (slot.Stage == SlotStage.Growing) slots[i] = (char)('1' + speciesIndex);
                else slots[i] = '0';
            }
            var blocks = new char[OrderTable.BlockCount];
            for (int b = 0; b &lt; blocks.Length; b++)
            {
                int[] blockSlots = OrderTable.GetBlockSlots(b);
                SlotState head = blockSlots == null ? null : state.Slots[blockSlots[0]];
                blocks[b] = head != null &amp;&amp; head.Renovated ? '1' : '0';
            }
            int petals = Math.Max(0, Math.Min(GardenSystem.WalletCap, state.Petals));
            return Prefix + "|" + new string(slots) + "|" + new string(blocks) + "|" + petals;
        }
```

```
/// &lt;summary&gt;Decode a visitor-typed share code into a playable GardenState; null on malformed input.&lt;/s
public static GardenState DecodeShare(string code)
{
    if (string.IsNullOrEmpty(code)) return null;
    string trimmed = code.Trim();
    if (trimmed.Length == 0 || trimmed.Length &gt; MaxShareChars) return null;
    string[] parts = trimmed.Split('|');
    if (parts.Length != Sections) return null;
    if (!string.Equals(parts[0], Prefix, StringComparison.Ordinal)) return null;
    string slotSection = parts[1];
    string blockSection = parts[2];
    string petalSection = parts[3];
    if (slotSection.Length != GardenSystem.SlotCount) return null;
    if (blockSection.Length != OrderTable.BlockCount) return null;
    var state = new GardenState();
    for (int i = 0; i &lt; slotSection.Length; i++)
    {
        char c = slotSection[i];
        if (c == '0') continue; // empty is the constructed default
        int speciesIndex;
        SlotStage stage;
        if (c &gt;= '1' &amp;&amp; c &lt;= '4') { speciesIndex = c - '1'; stage = SlotStage.Growing; }
        else if (c &gt;= 'A' &amp;&amp; c &lt;= 'D') { speciesIndex = c - 'A'; stage = SlotStage.Bloom; }
        else return null;
        SpeciesDef def = SpeciesTable.Get(speciesIndex);
        if (def == null) return null;
        state.Slots[i].SpeciesId = def.Id;
        state.Slots[i].Stage = stage;
        state.Slots[i].PlantedAt = 0; // visitor clock owns growth from now (living-garden entrance)
    }
    for (int b = 0; b &lt; blockSection.Length; b++)
    {
        char c = blockSection[b];
        if (c != '0' &amp;&amp; c != '1') return null;
        if (c != '1') continue;
        int[] blockSlots = OrderTable.GetBlockSlots(b);
        if (blockSlots == null) return null;
        for (int s = 0; s &lt; blockSlots.Length; s++)
            state.Slots[blockSlots[s]].Renovated = true;
    }
    if (petalSection.Length &lt; 1 || petalSection.Length &gt; 3) return null;
    for (int i = 0; i &lt; petalSection.Length; i++)
        if (petalSection[i] &lt; '0' || petalSection[i] &gt; '9') return null;
    int petals;
    if (!int.TryParse(petalSection, out petals)) return null;
    if (petals &lt; 0 || petals &gt; GardenSystem.WalletCap) return null;
    state.Petals = petals;
    return state;
}
/// &lt;summary&gt;Species roster index 0..3; -1 when the id is unknown or empty.&lt;/summary&gt;
```

```
        static int SpeciesIndexOf(string speciesId)
        {
            for (int i = 0; i &lt; SpeciesTable.Count; i++)
            {
                SpeciesDef def = SpeciesTable.Get(i);
                if (def != null &amp;&amp; string.Equals(def.Id, speciesId, StringComparison.Ordinal)) return i;
            }
            return -1;
        }
    }
}
// ShareView.cs
// G19 share page host (T006): S04_Share overlay - the ShareCodec consumer.
// Opens from the S01 share entry button. Shows the garden's live share code,
// copies it to the system clipboard, and decodes a friend's code into the
// visitor garden (living-garden entrance: growing slots keep PlantedAt=0
// so they bloom on the visitor clock). Malformed input never throws (cozy
// law) - the status face says the code was not recognized and play goes on.
// Copy stays English-only on the shipped G19Main pixel face (X1302 HUD
// font chain; a CJK arrival needs a census+subset re-cast, U184 risk noted).
using System;
using Sy.BloomHaven.Audio;
using Sy.BloomHaven.Boot;
using Sy.BloomHaven.Garden;
using Sy.BloomHaven.Localize;
using Sy.BloomHaven.UI;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.BloomHaven.Share
{
    public sealed class ShareView : MonoBehaviour
    {
        public const string PageRootName = "S04_Share"; // T006 page-list name
        public const int PageSortOrder = 2;              // above garden(0)/board(1), below guide(3)
        public const float ReferenceWidth = 720f;
        public const float ReferenceHeight = 1280f;
        // element contract (PagesCheck zero-missing law, requirement doc S04 face)
        public const string CodeFaceName = "ShareCodeFace";
        public const string CopyButtonName = "CopyButton";
        public const string DecodeFieldName = "DecodeField";
        public const string VisitButtonName = "VisitButton";
        public const string StatusFaceName = "ShareStatus";
        public const string CloseButtonName = "CloseButton";
        // S4 AIGC transparency note (T012-c, doc-19 sec-5): AI-assisted art and
        // audio declaration line at the share-page bottom.
        public const string AigcNoteName = "AigcNote";
        public const string AigcNoteKey = "share.aigc_note";
        // T011-b reskin contract: the share card frame (AR-06) backs the
```

```
// code column; art missing keeps the modal's dim backdrop.
public const string CardArtName = "ShareCardArt";
public const string CardArtKey = "share_card_frame";
// AR-09 (T012-d) HUD chrome keys for the card's action buttons:
// copy rides the share-domain pink chrome, visit/close their own.
public const string CopyChromeKey = "hud_btn_share";
public const string VisitChromeKey = "hud_btn_visit";
public const string CloseChromeKey = "hud_btn_close";
// X1062 contrast law: labels ride the chrome's light fill.
static readonly Color ChromeLabelColor = new Color(0.36f, 0.26f, 0.18f, 1f);
public const float CardArtWidth = 540f;  // 9:16 portrait of the 750x1334 spec
public const float CardArtHeight = 960f;
public const float CardArtY = 150f;      // centers the code column on the card
// U088 X1048 settle-frame evidence: text boxes wider than the card's
// inner panel run onto the ornate border (code at 24pt reached the
// gold trim; the 600px decode field overhung the 540px card frame).
// CardInnerWidth keeps every caption inside the trim; FieldWidth
// keeps the input bar inside the card frame.
public const float CardInnerWidth = 460f;
// U088 X1062 (defect b): the cream inner panel is narrower than the
// gold trim - at 400 the dark input bar still overlapped the flower
// border in the 1:1 native frame; 320 keeps it inside the cream face.
public const float FieldWidth = 320f;
// status law faces (cozy copy, assertable by name at PagesCheck;
// values resolve through cfg_i18n keys - T008 U114 L0)
public const string InvalidCodeKey = "share.invalid_code";
public const string VisitedKey = "share.visited";
public const string CopiedKey = "share.copied";
public static readonly string InvalidCodeMessage = LocalizeService.Get(InvalidCodeKey);
public static readonly string VisitedMessage = LocalizeService.Get(VisitedKey);
public static readonly string CopiedMessage = LocalizeService.Get(CopiedKey);
public const float VisitCloseDelaySeconds = 1.2f; // welcome line hold, then step in;
// counters for future play-mode gates (redline11-style assertions)
public int CopyTaps;
public int VisitTaps;
public int LastDecodeOk = -1; // -1 = no attempt yet, 0 = rejected, 1 = visited
public bool IsOpen;
GardenView _garden;
Text _codeFace;
Text _statusFace;
InputField _decodeField;
GameObject _panel;
bool _built;
float _autoCloseAt = -1f; // visit success: show the welcome line, then step in
static bool _subscribed;
/// &lt;summary&gt;
/// Wire the share page onto S19_Main as soon as it finishes loading
/// (same static-installer family as GardenView: the boot host is
/// destroyed by the single-mode switch before its coroutine resumes).
/// The panel starts hidden - S01 owns the screen until the entry tap.
```

```
/// &lt;/summary&gt;
public static void InstallOnSceneLoad()
{
    if (_subscribed) return;
    _subscribed = true;
    SceneManager.sceneLoaded += OnSceneLoaded;
}
static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneIds.Main) return;
    EnsureInstalled();
}
public static ShareView EnsureInstalled()
{
    ShareView view = (ShareView)FindObjectOfType(typeof(ShareView));
    if (view != null) return view;
    // input floor (U184 family lesson): programmatic UGUI taps die
    // without an EventSystem in the loaded scene.
    if (FindObjectOfType(typeof(EventSystem)) == null)
        new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
    var go = new GameObject(PageRootName, typeof(ShareView));
    return go.GetComponent&lt;ShareView&gt;();
}
/// &lt;summary&gt;Gate seam (PagesCheck group C): the code face always
/// renders exactly what the codec composes for the live garden.&lt;/summary&gt;
public static string ComposeCurrentCode(GardenState state)
{
    return ShareCodec.ComposeShare(state);
}
void Start() =&gt; Initialize();
/// &lt;summary&gt;Headless boot seam (U085 slice-B patrol): the full Start()
/// face callable from editor gates (see GardenView.Initialize).&lt;/summary&gt;
public void Initialize()
{
    _garden = GardenView.EnsureInstalled();
    BuildUi();
    SetOpen(false);
}
// ----- open/close -----
/// &lt;summary&gt;Entry toggle from the S01 share button.&lt;/summary&gt;
public void Toggle()
{
    SetOpen(!IsOpen);
}
void SetOpen(bool open)
{
    IsOpen = open;
    _autoCloseAt = -1f; // any manual open/close cancels a pending auto-close
    if (_panel == null) return;
    _panel.SetActive(open);
```

```
    if (_garden != null)
    {
        // U088 X1062 modal guards: (a) garden floats stay down so no
        // '+1'/'bluebell' label peeks from behind the card; (d) the
        // HUD share entry is fully hidden while the modal owns the
        // screen - half-occluded by the card corner was the defect.
        _garden.SetFloatsSuppressed(open);
        _garden.SetShareEntryVisible(!open);
    }
    if (open)
    {
        RefreshCode();
        if (_statusFace != null) _statusFace.text = string.Empty;
        if (_decodeField != null) _decodeField.text = string.Empty;
    }
}
void RefreshCode()
{
    if (_codeFace == null || _garden == null || _garden.System == null) return;
    _codeFace.text = ComposeCurrentCode(_garden.System.State);
}
// ----- interactions -----
void OnCopyTapped()
{
    CopyTaps++;
    if (_garden == null || _garden.System == null) return;
    string code = ComposeCurrentCode(_garden.System.State);
    if (string.IsNullOrEmpty(code)) return;
    GUIUtility.systemCopyBuffer = code;
    AudioHost.PlayShareHook(); // X1079: share-confirmation chime on the real copy path
    if (_statusFace != null) _statusFace.text = CopiedMessage;
}
void OnVisitTapped()
{
    VisitTaps++;
    if (_decodeField == null) return;
    GardenState visitor = ShareCodec.DecodeShare(_decodeField.text);
    if (visitor == null)
    {
        LastDecodeOk = 0;
        if (_statusFace != null) _statusFace.text = InvalidCodeMessage;
        return;
    }
    LastDecodeOk = 1;
    _garden.LoadVisitorGarden(visitor); // in-place copy, board stays live
    if (_statusFace != null) _statusFace.text = VisitedMessage;
    _autoCloseAt = Time.unscaledTime + VisitCloseDelaySeconds; // welcome line first, then step in
}
void Update()
{
```

```
    if (IsOpen &amp;&amp; _autoCloseAt &gt;= 0f &amp;&amp; Time.unscaledTime &gt;= _autoCloseAt)
        SetOpen(false);
}
void OnCloseTapped()
{
    SetOpen(false);
}
// ----- UI construction (programmatic, zero scene authoring) -----
void BuildUi()
{
    if (_built) return;
    _built = true;
    var canvasGo = new GameObject("ShareCanvas", typeof(Canvas), typeof(CanvasScaler),
        typeof(GraphicRaycaster));
    canvasGo.transform.SetParent(transform, false); // page root owns the canvas
    var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    canvas.sortingOrder = PageSortOrder; // overlay law: garden 0 -&gt; board 1 -&gt; share 2
    var scaler = canvasGo.GetComponent&lt;CanvasScaler&gt;();
    // U184: ScaleWithScreenSize or the reference frame never drives
    // device scale (raw ConstantPixelSize = shrunken UI on phone).
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(ReferenceWidth, ReferenceHeight);
    _panel = new GameObject("SharePanel", typeof(RectTransform), typeof(Image));
    _panel.transform.SetParent(canvasGo.transform, false);
    var panel = (RectTransform)_panel.transform;
    panel.anchorMin = Vector2.zero;
    panel.anchorMax = Vector2.one;
    panel.sizeDelta = Vector2.zero; // full-screen dim backdrop
    panel.anchoredPosition = Vector2.zero;
    var backdrop = _panel.GetComponent&lt;Image&gt;();
    backdrop.color = new Color(0.08f, 0.06f, 0.10f, 0.82f);
    backdrop.raycastTarget = true; // modal: swallow taps behind the panel
    // T011-b: the share card frame (AR-06) backs the code column,
    // null-safe - art missing keeps the dim-backdrop face.
    Sy.BloomHaven.UI.PageArt.BuildNode(_panel.transform, CardArtName, 0f, CardArtY,
        CardArtWidth, CardArtHeight, Sy.Core.ArtRuntime.Get(CardArtKey));
    var title = MakeText(_panel.transform, "ShareTitle", 32f, new Vector2(0f, 430f), 680, 44);
    // U088 X1062 (defect c): the title rides the card's cream face -
    // deep warm brown instead of white-on-cream low contrast.
    title.color = new Color(0.36f, 0.26f, 0.18f, 1f);
    title.text = LocalizeService.Get("share.title");
    var codeLabel = MakeText(_panel.transform, "CodeLabel", 18f, new Vector2(0f, 372f), (int)CardInnerWidth, 28)
    codeLabel.color = new Color(0.40f, 0.30f, 0.22f, 1f); // U088 X1062: brown on cream, not white-on-cream
    codeLabel.text = LocalizeService.Get("share.code_label");
    // U088 X1048: 24pt pushed the code past the card's gold trim and
    // rasters with a 1px double-strike; 20pt fits the inner panel
    // (worst case BH1|16 slots|4 blocks|999 = 29 chars). Box 88 tall
    // at y=308 keeps the label band (358..386) collision-free.
    _codeFace = MakeText(_panel.transform, CodeFaceName, 20f, new Vector2(0f, 308f), (int)CardInnerWidth, 88);
```

```
        _codeFace.color = new Color(0.13f, 0.28f, 0.46f, 1f); // U088 X1062: deep ink blue on cream (was pale blue)
        _codeFace.horizontalOverflow = HorizontalWrapMode.Wrap;
        var copy = MakeButton(_panel.transform, CopyButtonName, "Copy code", 0f, 228f, 190f, 52f);
        copy.image.color = new Color(0.36f, 0.52f, 0.42f, 1f);
        ApplyChrome(copy, CopyChromeKey); // AR-09: share-domain pink
        copy.onClick.AddListener(OnCopyTapped);
        var visitLabel = MakeText(_panel.transform, "VisitLabel", 18f, new Vector2(0f, 150f), (int)CardInnerWidth, 2
        visitLabel.color = new Color(0.40f, 0.30f, 0.22f, 1f); // U088 X1062: brown on cream, not white-on-cream
        visitLabel.text = LocalizeService.Get("share.visit_label");
        _decodeField = MakeDecodeField(_panel.transform, 88f);
        _decodeField.text = string.Empty;
        var visit = MakeButton(_panel.transform, VisitButtonName, "Visit", 0f, 18f, 190f, 52f);
        visit.image.color = new Color(0.42f, 0.56f, 0.86f, 1f);
        ApplyChrome(visit, VisitChromeKey); // AR-09: visitor blue
        visit.onClick.AddListener(OnVisitTapped);
        _statusFace = MakeText(_panel.transform, StatusFaceName, 18f, new Vector2(0f, -42f), (int)CardInnerWidth, 30
        _statusFace.color = new Color(1f, 0.85f, 0.45f, 1f);
        _statusFace.text = string.Empty;
        var close = MakeButton(_panel.transform, CloseButtonName, "Close", 0f, -110f, 140f, 46f);
        close.image.color = new Color(0.42f, 0.36f, 0.36f, 1f);
        ApplyChrome(close, CloseChromeKey); // AR-09: warm red
        close.onClick.AddListener(OnCloseTapped);
        // S4 AIGC transparency note (T012-c): doc-19 sec-5 declaration line.
        // Art and audio assets are AI-generated; the note rides the share
        // page (the game's community surface), value via cfg_i18n key.
        var aigcNote = MakeText(_panel.transform, AigcNoteName, 13f, new Vector2(0f, -168f), 680, 24);
        aigcNote.color = new Color(0.42f, 0.34f, 0.27f, 0.90f); // U088 X1062: muted brown, readable on cream
        aigcNote.text = LocalizeService.Get(AigcNoteKey);
    }
    /// &lt;summary&gt;AR-09 (T012-d): apply the HUD chrome sprite to a button
    /// face, null-safe - art missing keeps the procedural tint (X027
    /// seam). The label goes deep warm brown for the chrome's light fill.&lt;/summary&gt;
    static void ApplyChrome(Button button, string key)
    {
        Sprite chrome = Sy.Core.ArtRuntime.Get(key);
        if (chrome == null) return;
        button.image.sprite = chrome;
        button.image.color = Color.white; // sprite carries its own hue
        var label = button.GetComponentInChildren&lt;Text&gt;();
        if (label != null) label.color = ChromeLabelColor;
    }
    /// &lt;summary&gt;"Paste a friend's code" input line: background image +
    /// placeholder + text component, per UGUI InputField requirements.&lt;/summary&gt;
    InputField MakeDecodeField(Transform parent, float y)
    {
        var fieldGo = new GameObject(DecodeFieldName, typeof(RectTransform), typeof(Image), typeof(InputField));
        fieldGo.transform.SetParent(parent, false);
        var rt = (RectTransform)fieldGo.transform;
        rt.sizeDelta = new Vector2(FieldWidth, 64f); // U088 X1048: inside the card frame
        rt.anchoredPosition = new Vector2(0f, y);
```

```
        var bg = fieldGo.GetComponent&lt;Image&gt;();
        bg.color = new Color(0.16f, 0.14f, 0.20f, 0.95f);
        var field = fieldGo.GetComponent&lt;InputField&gt;();
        field.targetGraphic = bg; // U184: real target graphic for tint states
        var textGo = new GameObject("TextField", typeof(RectTransform), typeof(Text));
        textGo.transform.SetParent(fieldGo.transform, false);
        var textRt = (RectTransform)textGo.transform;
        textRt.anchorMin = new Vector2(0f, 0f);
        textRt.anchorMax = new Vector2(1f, 1f);
        textRt.sizeDelta = new Vector2(-24f, -14f);
        textRt.anchoredPosition = new Vector2(12f, 0f);
        var text = textGo.GetComponent&lt;Text&gt;();
        text.font = HudFont.Resolve();
        text.fontSize = 22;
        text.alignment = TextAnchor.MiddleLeft;
        text.color = Color.white;
        text.supportRichText = false; // pasted codes are plain text, never markup
        text.raycastTarget = false;
        field.textComponent = text;
        var placeholderGo = new GameObject("Placeholder", typeof(RectTransform), typeof(Text));
        placeholderGo.transform.SetParent(fieldGo.transform, false);
        var placeholderRt = (RectTransform)placeholderGo.transform;
        placeholderRt.anchorMin = new Vector2(0f, 0f);
        placeholderRt.anchorMax = new Vector2(1f, 1f);
        placeholderRt.sizeDelta = new Vector2(-24f, -14f);
        placeholderRt.anchoredPosition = new Vector2(12f, 0f);
        var placeholder = placeholderGo.GetComponent&lt;Text&gt;();
        placeholder.font = HudFont.Resolve();
        placeholder.fontSize = 20;
        placeholder.alignment = TextAnchor.MiddleLeft;
        placeholder.color = new Color(1f, 1f, 1f, 0.40f);
        placeholder.text = LocalizeService.Get("share.placeholder");
        placeholder.raycastTarget = false;
        field.placeholder = placeholder;
        return field;
    }
    static Button MakeButton(Transform parent, string name, string label,
        float x, float y, float w, float h)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.sizeDelta = new Vector2(w, h);
        rt.anchoredPosition = new Vector2(x, y);
        var button = go.GetComponent&lt;Button&gt;();
        button.targetGraphic = go.GetComponent&lt;Image&gt;(); // U184: real target graphic
        button.transition = Selectable.Transition.ColorTint;
        var text = MakeText(go.transform, "Label", 16f, Vector2.zero, (int)w - 6, (int)h - 8);
        text.text = label;
        return button;
```

```
        }
        static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.sizeDelta = new Vector2(w, h);
            rt.anchoredPosition = pos;
            var t = go.GetComponent&lt;Text&gt;();
            t.font = HudFont.Resolve();
            t.fontSize = Mathf.RoundToInt(size);
            t.alignment = TextAnchor.MiddleCenter;
            t.color = Color.white;
            t.raycastTarget = false; // never steal a button tap
            return t;
        }
    }
}
// HudFont.cs
// Pixel-font HUD chain (P2 advisory X1070, FT-011 subset X1301): the
// shipped G19Main face (Fusion Pixel 12px subset, census-covered) is the
// single HUD font seam - every player-facing Text face resolves through
// here. A missing or pruned asset falls back to the builtin face so a
// label never renders dead (X027 null-safe law); the shipped census
// (Design/configs/G19/G19_font_census.txt) covers every i18n value and
// code literal, so the subset can never tofu before I18nCheck group C
// fails the build first.
using UnityEngine;
namespace Sy.BloomHaven.UI
{
    public static class HudFont
    {
        const string ResourcePath = "Fonts/G19Main";
        const string BuiltinFallback = "LegacyRuntime.ttf";
        static Font _cached;
        /// &lt;summary&gt;HUD font face. Cached on first resolve; falls back to
        /// the builtin face when the shipped subset is absent so the UI
        /// never depends on a live asset to render (X027 seam).&lt;/summary&gt;
        public static Font Resolve()
        {
            if (_cached != null) return _cached;
            var font = Resources.Load&lt;Font&gt;(ResourcePath);
            if (font == null) font = Resources.GetBuiltinResource&lt;Font&gt;(BuiltinFallback);
            _cached = font;
            return _cached;
        }
    }
}
// PageArt.cs
// T011-b page-art wiring helper: the three reskinned pages (S02 board,
```

```
// S03 codex, S04 share) build one art node each and apply the produced
// sprite through this seam so the gate can assert the null-safe law in
// one place. Art missing (catalog empty, art pruned, name typo) means the
// node sleeps and the procedural color-block face stays untouched - the
// same contract ArtRuntime.Get already enforces for every consumer.
using UnityEngine;
using UnityEngine.UI;
namespace Sy.BloomHaven.UI
{
    public static class PageArt
    {
        /// &lt;summary&gt;
        /// Build the page-art node (anchored center, behind every sibling)
        /// and apply the sprite. Returns the node whether or not art lives.
        /// &lt;/summary&gt;
        public static GameObject BuildNode(Transform parent, string name,
            float x, float y, float w, float h, Sprite art)
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.sizeDelta = new Vector2(w, h);
            rt.anchoredPosition = new Vector2(x, y);
            rt.SetAsFirstSibling(); // page art sits behind every sibling face
            Apply(go.GetComponent&lt;Image&gt;(), art);
            return go;
        }
        /// &lt;summary&gt;
        /// Null-safe sprite apply. Returns true when the node shows live art;
        /// a null sprite deactivates the node so the procedural look stays.
        /// &lt;/summary&gt;
        public static bool Apply(Image target, Sprite art)
        {
            if (target == null) return false;
            if (art == null)
            {
                target.sprite = null;
                target.color = Color.white;
                target.gameObject.SetActive(false);
                return false;
            }
            target.sprite = art;
            target.color = Color.white;
            target.preserveAspect = true;
            target.raycastTarget = false; // backdrop never steals a tap
            target.gameObject.SetActive(true);
            return true;
        }
    }
}
```